國家實驗研究院
國家高速網路與計算中心
National Center for High-performance Computing

NVIDIA PhysicsNeMo for NCHC AI-Powered Physics Bootcamp

Chao-Shun Zhan | Solutions Architect, Mar. 2026

Day 1 (3/10)

- Overview of PhysicsNeMo
- PhysicsNeMo Architecture, Traning
- Hands-on with PhysicsNeMo — tutorial
  - Example1: Projectile Motion ODE
  - Example2: 1D Diffusion
  - Example3: Forecasting Weather using Navier Stokes PDE
- Day 2 Reminder

Day 2 (3/11)

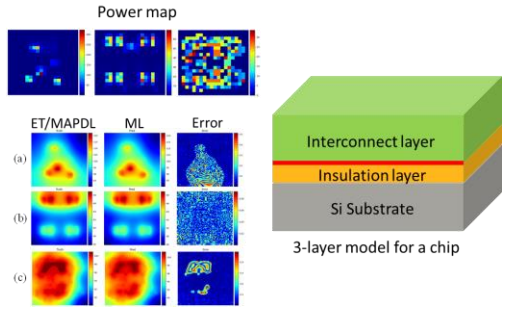
- Earth2 Overview
- DoMINO Overview
- Sharing Session with Dr. Heng-Chuan Kan, NCHC
- Environment setting
- Lunch Break
- Hands-on : Challenge overview
  - Challenge1: Advanced Wave Dynamics
  - Challenge2: Fluid-Structure Interaction
  - Challenge3: Multi-Physics Climate Modeling
  - Challenge4: Advanced Neural Operators
- Announcing the Challenge Results and Final Scores.

Agenda

- Physics AI across different applications
- Why Physics AI for Engineering and Scientific simulations?
- What is PhysicsNeMo?
- Deep Learning Overview
- PhysicsNeMo Architecture, Traning
- Hands-on with PhysicsNeMo

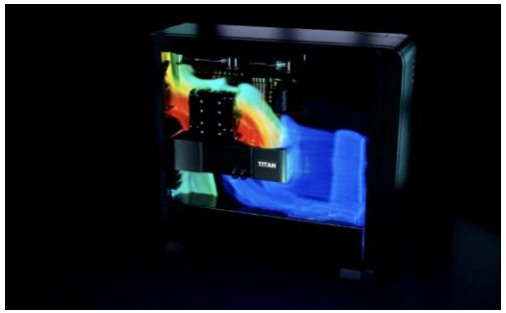
Applications

PhysicsNeMo Case-Studies: Physics AI across different applications



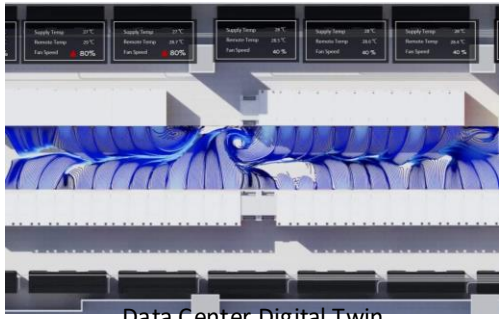
Electro-thermal cooling

Blog: Link



RTX 4090 heat sink design

Demo: Link

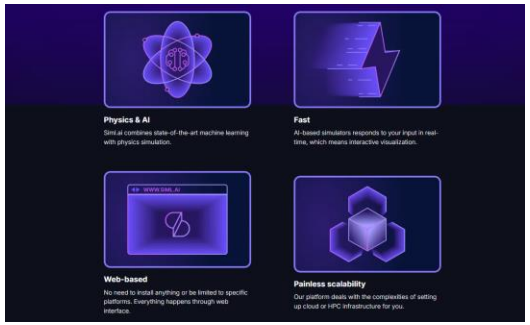


Data Center Digital Twin

Blog: Link, GTC Session: Link

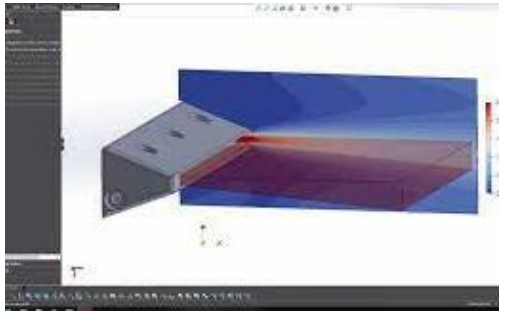
Automotive CFD

GTC 2024 Keynote

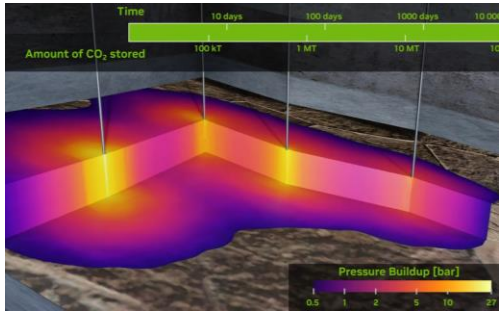


Siml.ai

Case Study: Link, Blog: Link

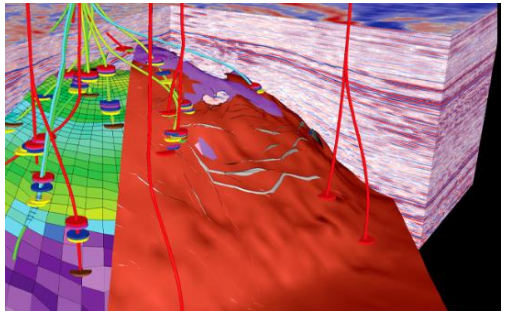


Design optimization



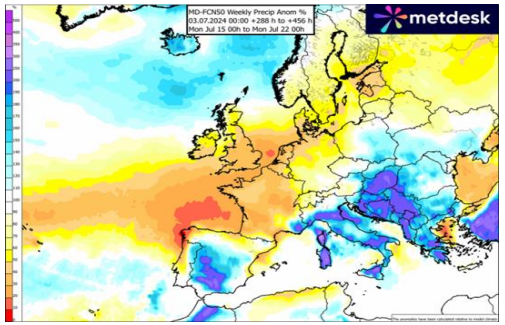
Carbon capture and storage

Demo: Link, Blog: Link



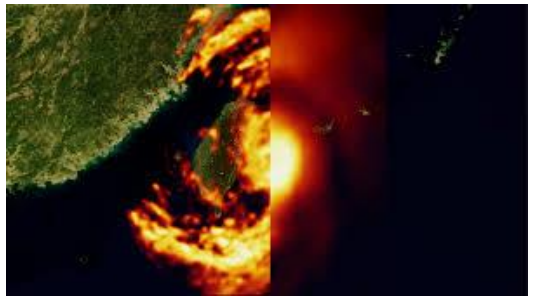
Sub surface simulations

Resource: Link



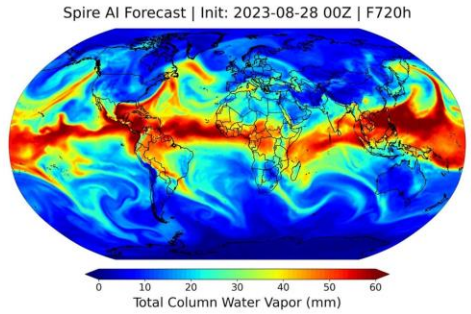
Energy Trading

Blog: Link



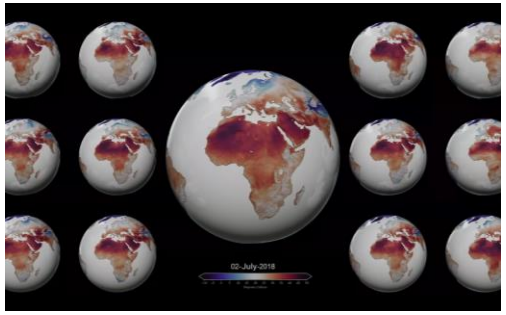
TWC: Weather modeling

Blog: Link



Spire: Weather modeling

Blog: Link

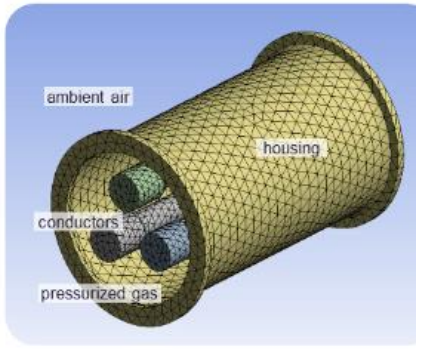


Meteomatics: Weather modeling

Blog: Link

NVIDIA

PhysicsNeMo Case-Studies: Physics AI across different applications

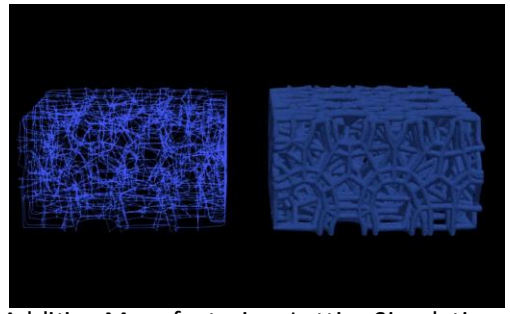


HRSG Digital Twin

Blog: Link, GTC Session: Link

HRSG Digital Twin

Demo: Link, GTC Session: Link

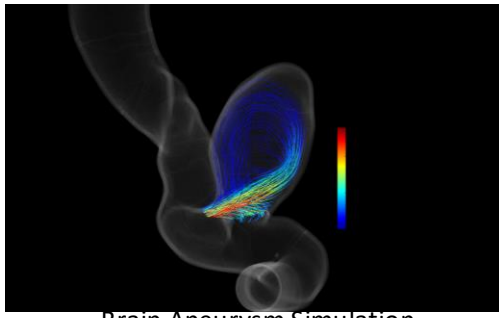


Additive Manufacturing: Lattice Simulation

Blog: Link

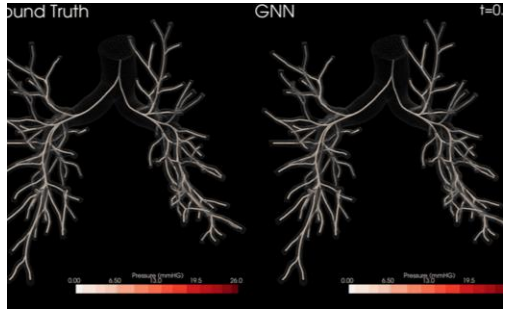
Additive Manufacturing: 3D Printing

Blog: Link



Brain Aneurysm Simulation

Demo: Link



Cardiovascular Simulation

Blog: Link

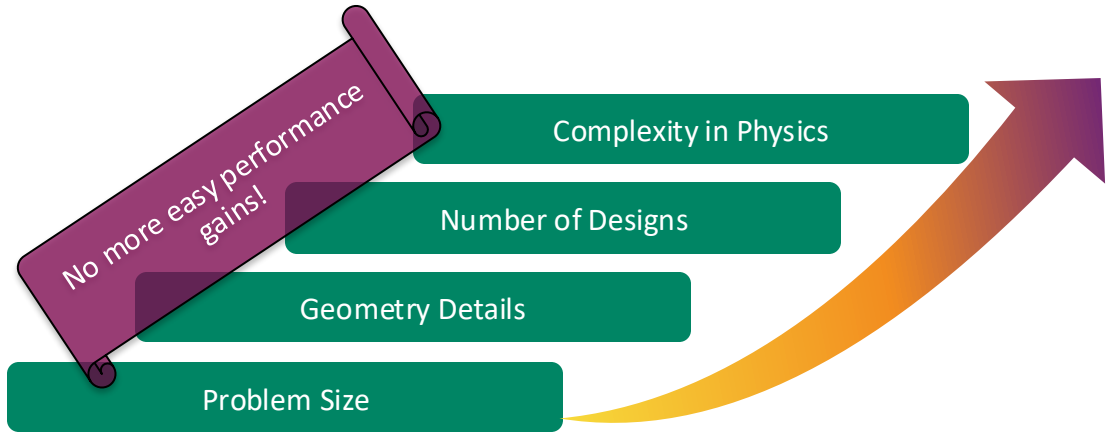
Wind farm Super Resolution

Demo: Link, Blog: Link

Why Physics AI for Engineering and Scientific simulations?

Simulations are a critical for engineering design

Simulations via numerical methods are computationally expensive



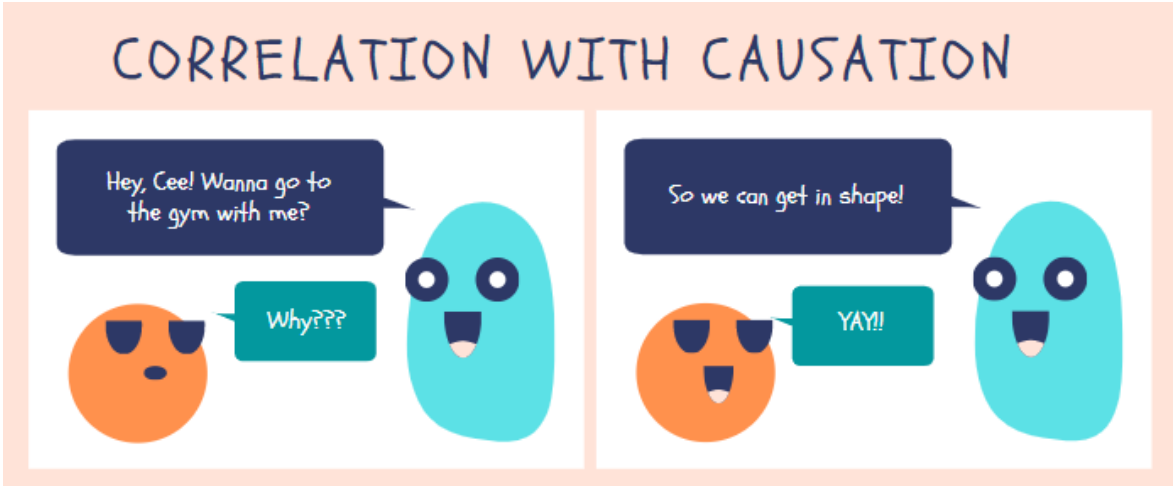
How can we expand use of simulations to create better product designs or better operational digital twins?

- Require near real time and high-fidelity simulation capabilities
- Require ability to ingest data coming from sensors or prior simulations

AI/ML for Physical systems

What is different?

- Developing an off the shelf surrogate model for users
  - Lack of generalizability
  - Dataset needs - data is limited and expensive
  - Satisfy the governing principles – coupled PDEs
  - CV based AI approaches not good enough - why?
    - These networks have convergence issues
      - Spectral bias: bias toward low-frequency solutions.
  - Scale of the problems



- **Physics-ML:** Physics guided / knowledge guided machine learning

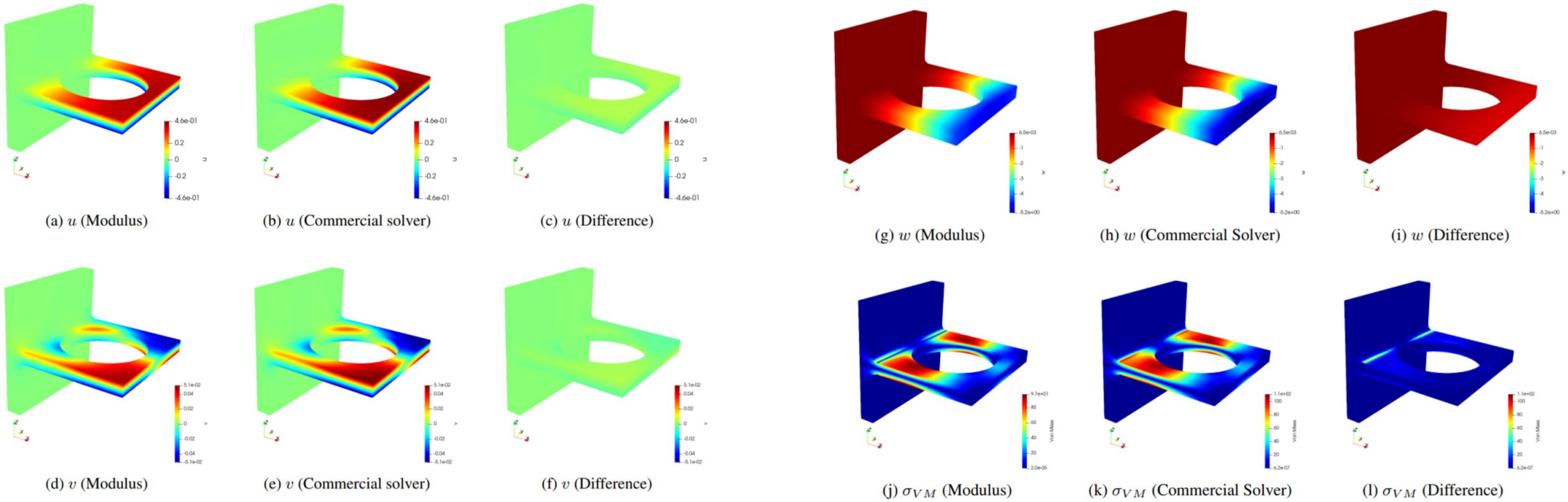
Integrating Scientific Knowledge with Machine Learning for Engineering and Environmental Systems).

PhysicsNeMo

What is PhysicsNeMo?

PhysicsNeMo is a PDE solver

- Like traditional solvers such as Finite Element, Finite Difference, Finite Volume, and Spectral solvers, PhysicsNeMo can solve PDEs



(a) u (Modulus) (b) u (Commercial solver) (c) u (Difference)

(g) w (Modulus) (h) w (Commercial Solver) (i) w (Difference)

(d) v (Modulus) (e) v (Commercial solver) (f) v (Difference)

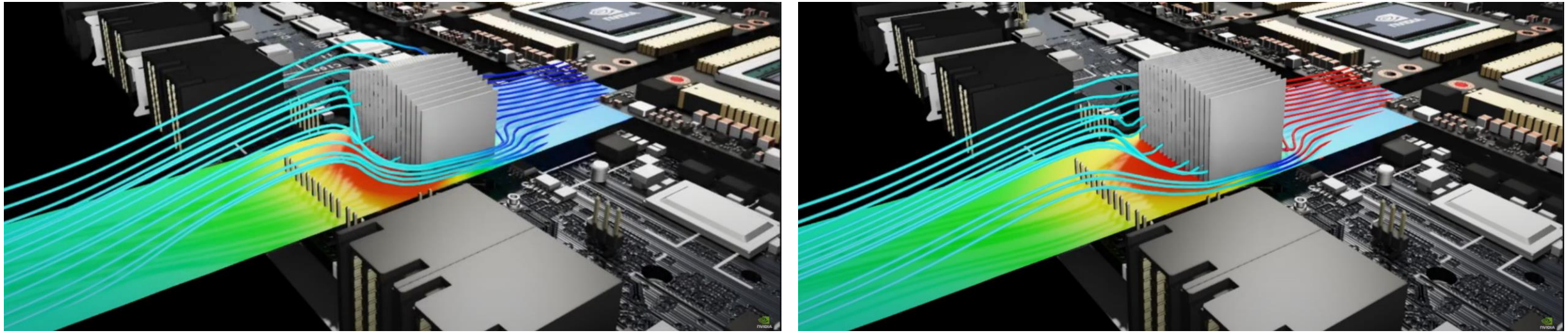
(j) σ_{VM} (Modulus) (k) σ_{VM} (Commercial Solver) (l) σ_{VM} (Difference)

A comparison between PhysicsNeMo and commercial solver results for a bracket deflection example. Linear elasticity equations are solved here.

What is PhysicsNeMo?

PhysicsNeMo is a tool for efficient design optimization for multi-physics problems

- With PhysicsNeMo, professionals in Manufacturing and Product Development can explore different configurations and scenarios of a model, in near-real time by, changing its parameters, allowing them to gain deeper insights about the system or product, and to perform efficient design optimization of their products.

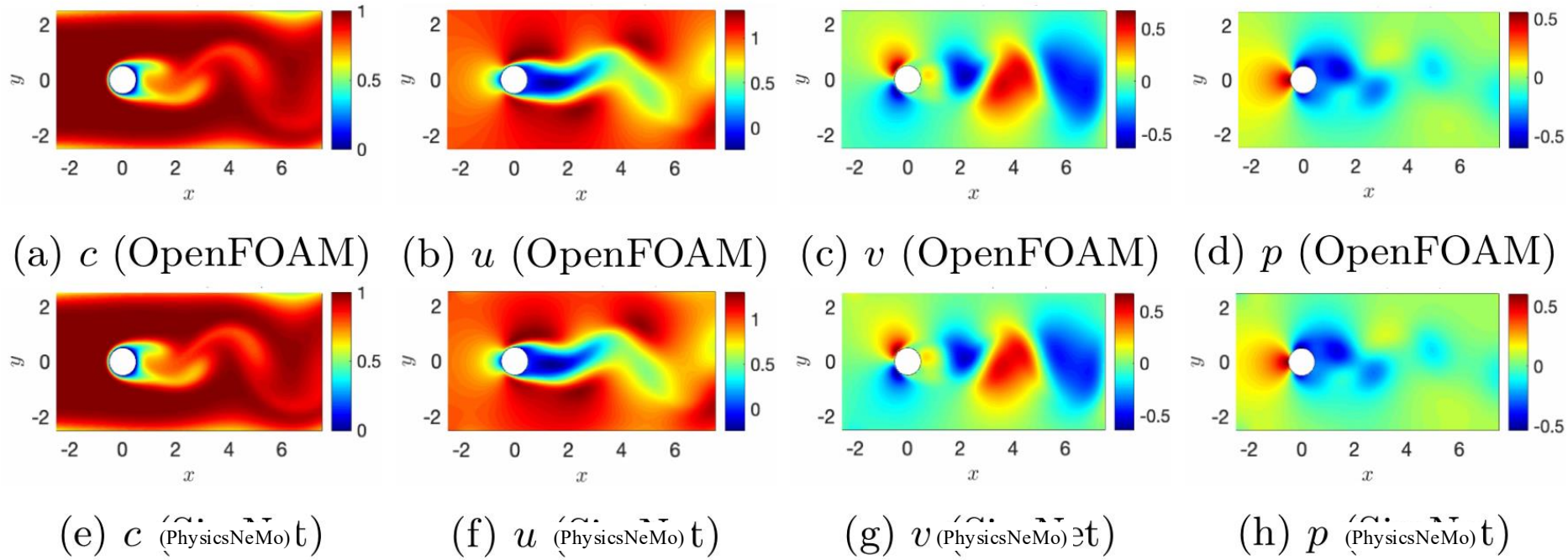


Efficient design space exploration of the heatsink of a Field-Programmable Gate Array (FPGA) using PhysicsNeMo.

What is PhysicsNeMo?

PhysicsNeMo is a solver for inverse problems

- Many applications in science and engineering involve inferring unknown system characteristics given measured data from sensors or imaging.
- By combining data and physics, PhysicsNeMo can effectively solve inverse problems.



A comparison between PhysicsNeMo and OpenFOAM results for the flow velocity, pressure and passive scalar concentration fields. PhysicsNeMo has inferred the velocity and pressure fields using scattered data from passive scalar concentration.

What is PhysicsNeMo?

PhysicsNeMo is a tool for developing data-driven solutions to engineering problems

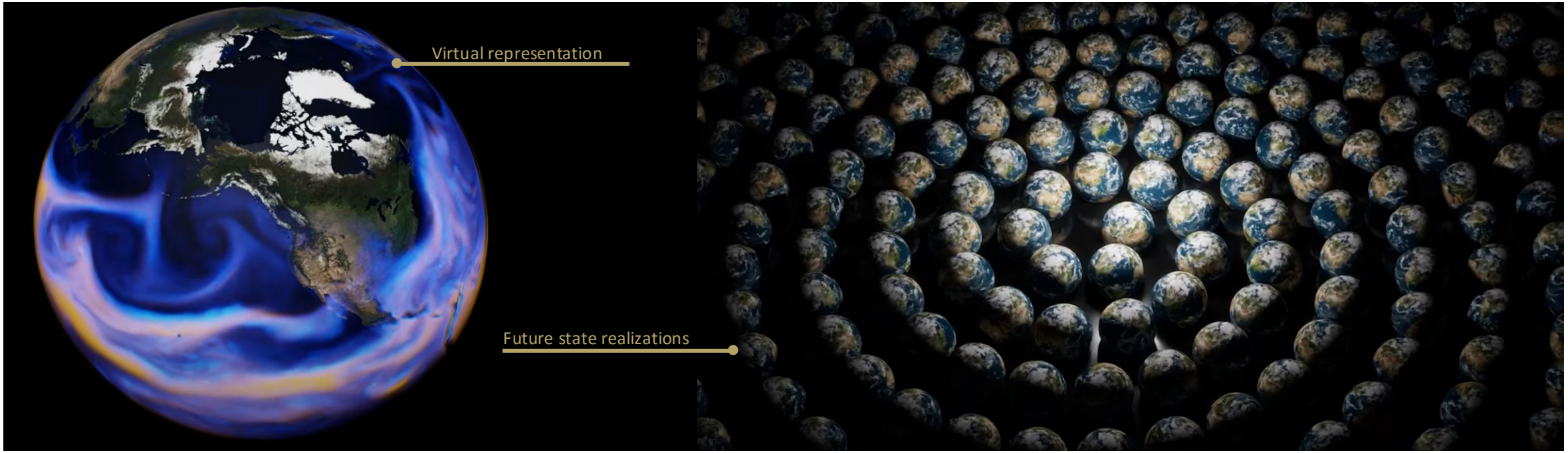
- PhysicsNeMo contains a variety of APIs for developing data-driven machine learning solutions to challenging engineering systems, including:
  - Data-driven modeling of physical systems
  - Super resolution of low-fidelity results computed by traditional solvers

Super-resolution of flow in a wind farm using PhysicsNeMo

What is PhysicsNeMo?

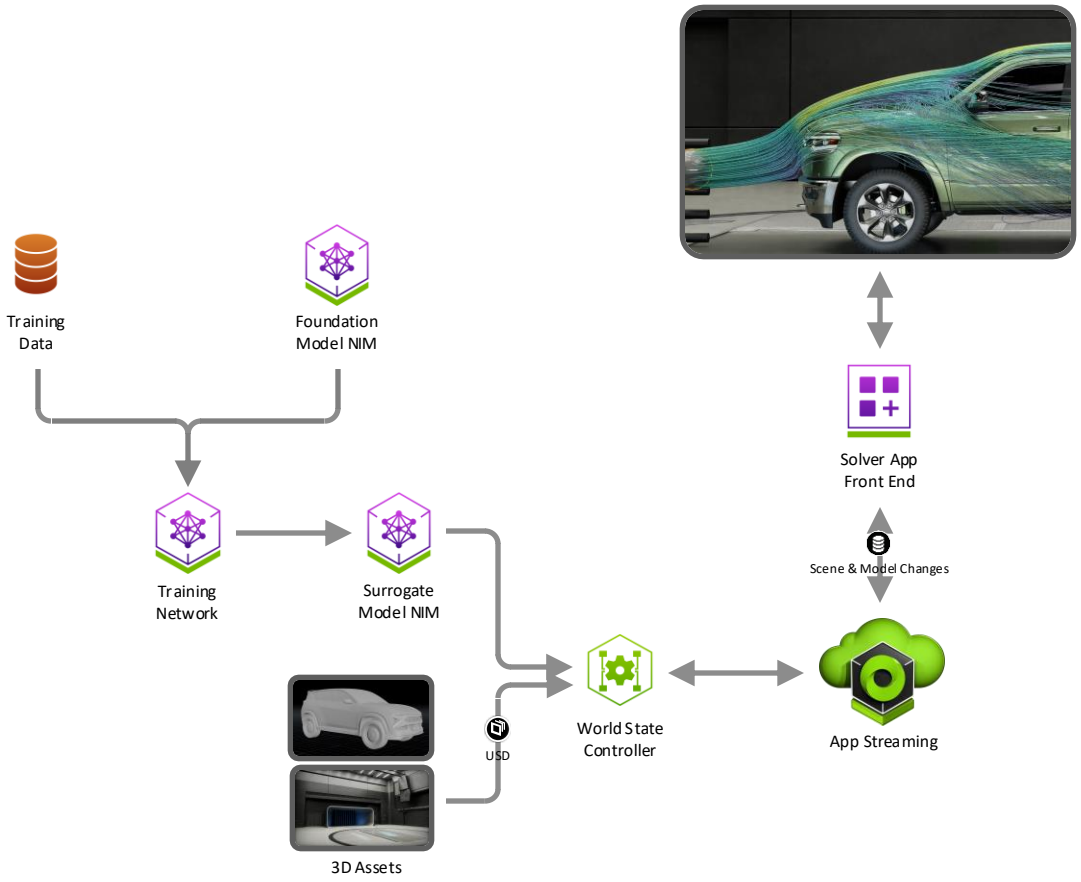
PhysicsNeMo is a tool for developing digital twins

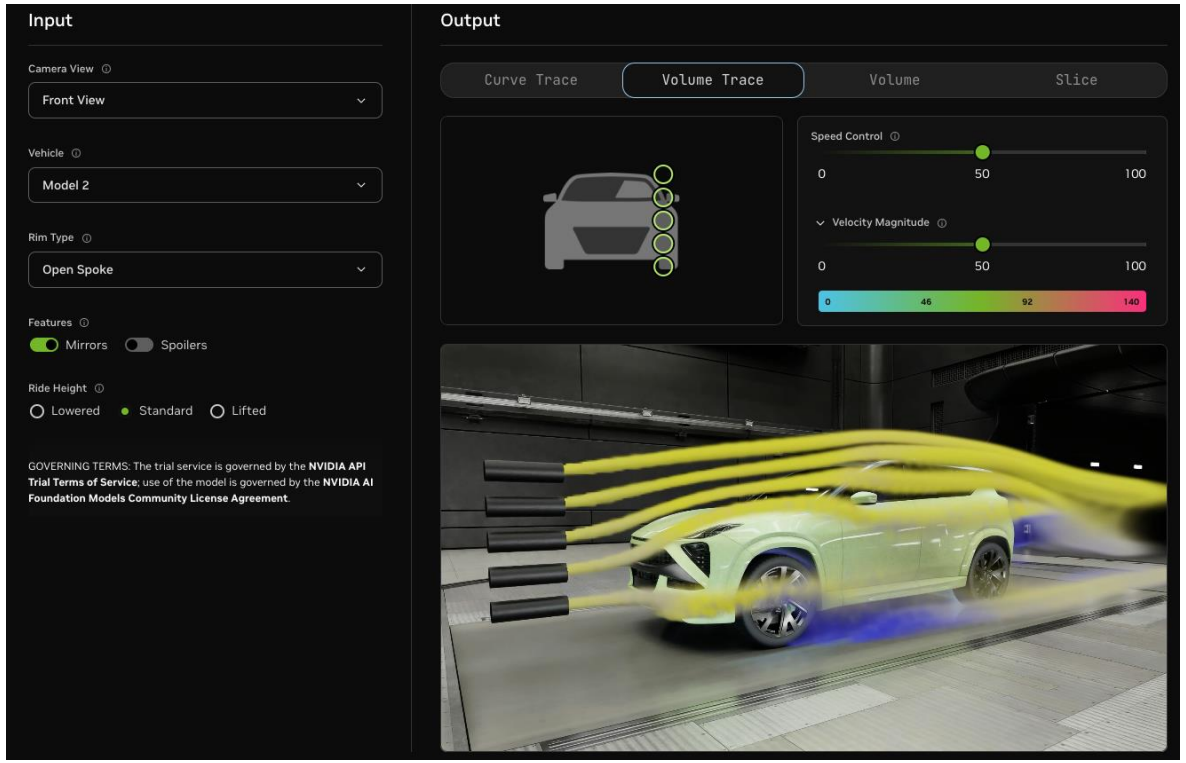
- A digital twin is a virtual representation (a true-to-reality simulation of physics) of a real-world physical asset or system, which is continuously updated via stream of data.
- Digital twin predicts the future state of the real-world system under varying conditions.



Omniverse Blueprint for Real-Time CAE Digital Twins

https://build.nvidia.com/nvidia/digital-twins-for-fluid-simulation





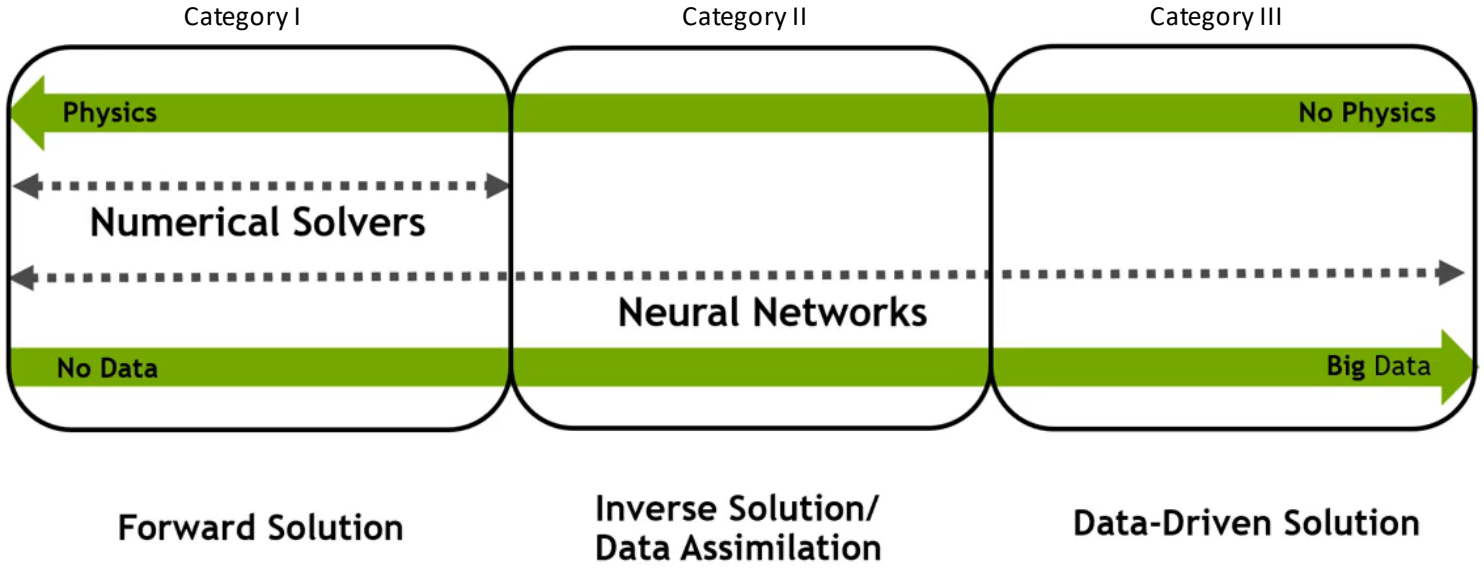
Ansys ran Fluent at the Texas Advanced Computing Center on 320 NVIDIA GH200 Grace Hopper Superchips. A 2.5-billion-cell automotive simulation was completed in just over six hours, which would have taken nearly a month running on 2,048 x86 CPU cores, significantly enhancing the feasibility of overnight high-fidelity CFD analyses and establishing a new industry benchmark.

What is PhysicsNeMo?

Putting it all together

- PhysicsNeMo is a PDE solver (category I)
- PhysicsNeMo is a tool for efficient design optimization & design space exploration (category I)
- PhysicsNeMo is a solver for inverse problems (category II)
- PhysicsNeMo is a tool for developing digital twins (category II)
- PhysicsNeMo is a tool for developing AI solutions to engineering problems (category III)

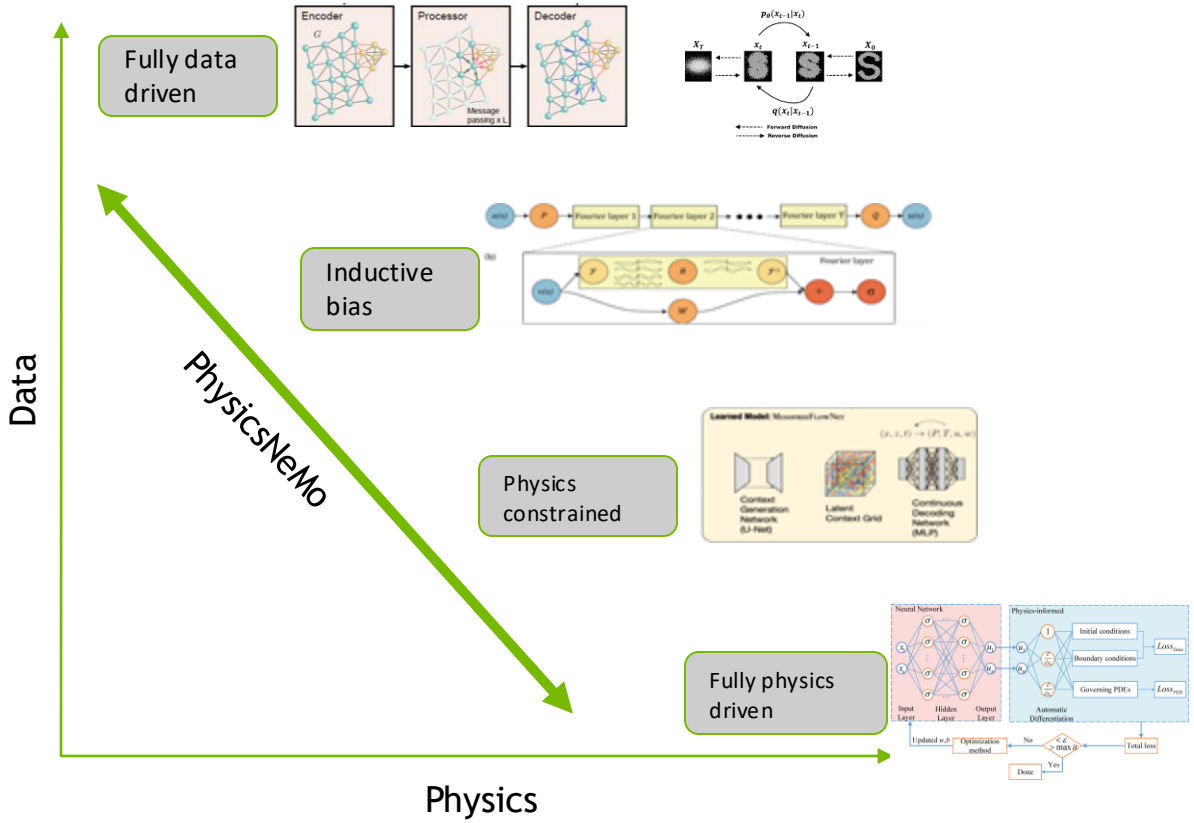
These are all done by developing deep neural network models in PhysicsNeMo that are physics-informed and/or data-informed.



Tailored for developing Physics AI models

Reference models with architectures tuned for Physics AI

- Bringing novel AI architectures that have demonstrated success for engineering and science problems
- Using case studies as reference starting points



Diverse Physics AI approaches:

- fully Physics driven AI modeling
- fully data driven AI modeling
- hybrid (data + Physics) AI modeling

Neural Operators:

- Fourier Neural Operator family (FNO, AFNO, PINO)
- DeepONet

GNNs:

- GraphCast
- MeshGraphNet ..

Diffusion Models:

- DDPM++
- NCSN++
- ADM ..

PDE informed Neural Networks:

- Fourier Feature Network
- Spatial-temporal Fourier Feature Networks
- Super Resolution Net …

FNO Performance benchmarking

Single GPU and multi-GPU

Model (# of Parameters)	Memory utilization(Gb) / Time taken(sec) per epoch			
	Batch size			
	1	2	3	4
PhysicsNeMo FNO (2.36m)	10.84/81.94	13.12/90.84	OOM	OOM
PhysicsNeMo FNO (1.33m)	10.62/64.1	12.75/74.1	14.04/72.9	OOO
PhysicsNeMo FNO (0.5m)	9.89/43	11.09/51	12.01/49.1	13.17/47.1
Base FNO (699k)	12.44/101.8	OOM	OOM	OOM
Base FNO (393k)	11.62/75.26	14.41/83.15	OOM	OOM

6X more params

2X batches with lower time

- Key takeaways:
  - PhysicsNeMo implemented FNO can accommodate a larger batch size for the same number of parameters. The training time per epoch is also better as compared to the Base FNO implementation.
  - PhysicsNeMo FNO can accommodate 6x larger number of model parameters for the same memory utilization.
- Low level kernels (FFT kernels, Vectorized and elementwise kernels etc.) evaluation is optimized in PhysicsNeMo implemented FNO .
- Minimal tensor copying resulting in better memory utilization.

Reference sample catalogue

https://github.com/NVIDIA/PhysicsNeMo/tree/main/examples

https://github.com/NVIDIA/PhysicsNeMo-sym/tree/main/examples

CFD

Use case	Model	Transient
Vortex Shedding	MeshGraphNet	YES
Ahmed Body Drag prediction	MeshGraphNet	NO
Navier-Stokes Flow	RNN	YES
Gray-Scott System	RNN	YES
Darcy Flow	FNO	NO
Darcy Flow using Nested-FNOs	Nested-FNO	NO
Darcy Flow (Data + Physics Driven) using DeepONet approach	FNO (branch) and MLP (trunk)	NO
Darcy Flow (Data + Physics Driven) using PINO approach (Numerical gradients)	FNO	NO
Stokes Flow (Physics Informed Fine-Tuning)	MeshGraphNet and MLP	NO

Weather

Use case	Model	AMP	CUDA Graphs	Multi-GPU	Multi-Node
Medium-range global weather forecast using FCN-SFNO	FCN-SFNO	YES	NO	YES	YES
Medium-range global weather forecast using GraphCast	GraphCast	YES	NO	YES	YES
Medium-range global weather forecast using FCN-AFNO	FCN-AFNO	YES	YES	YES	YES
Medium-range and S2S global weather forecast using DLWP	DLWP	YES	YES	YES	YES

Healthcare

Use case	Model	Transient
Cardiovascular Simulations	MeshGraphNet	YES
Brain Anomaly Detection	FNO	YES

Molecular Dymanics

Use case	Model	Transient
Force Prediciton for Lennard Jones system	MeshGraphNet	NO

Generative

Use case	Model	Multi-GPU	Multi-Node
Generative Correction Diffusion Model for Km-scale Atmospheric Downscaling	CorrDiff	YES	YES

Additional examples

In addition to the examples in this repo, more Physics-ML usecases and examples can be referenced from the Modulus-Sym examples.

Introductory

Use case	Model	Level	Attributes
Lid Driven Cavity Flow	Fully Connected MLP PINN	Introductory	Steady state, Multi-GPU
Anti-derivative	Data and Physics informed DeepONet	Introductory	Steady state, Multi-GPU
Darcy Flow	FNO, AFNO, PINO	Introductory	Steady state, Multi-GPU
Spring-mass system ODE	Fully Connected MLP PINN	Introductory	Steady state, Multi-GPU
Surface PDE	Fully Connected MLP PINN	Introductory	Steady state, Multi-GPU

Turbulence

Use case	Model	Level	Attributes
Taylor-Green	Fully Connected MLP PINN	Intermediate	Steady state, Multi-GPU
Turbulent channel	Fourier Feature MLP PINN	Intermediate	Steady state, Multi-GPU
Turbulent super-resolution	Super Resolution Network, Pix2Pix	Intermediate	Steady state, Multi-GPU

Electromagnetics

Use case	Model	Level	Attributes
Waveguide	Fourier Feature MLP PINN	Intermediate	Steady state, Multi-GPU

Solid Mechanics

Use case	Model	Level	Attributes
Plane displacement	Fully Connected MLP PINN, VPINN	Intermediate	Steady state, Multi-GPU

Design Optimization

Use case	Model	Level	Attributes
2D Chip	Fully Connected MLP PINN	Advanced	Steady state, Multi-GPU
3D Three fin Heatsink	Fully Connected MLP PINN	Advanced	Steady state, Multi-Node
FPGA Heatsink	Multiple Models (including Fourier Feature MLP PINN, SIREN:S, etc.)	Advanced	Steady state, Multi-Node
Limerock Industrial Heatsink	Fourier Feature MLP PINN	Advanced	Steady state, Multi-Node

Geophyscis

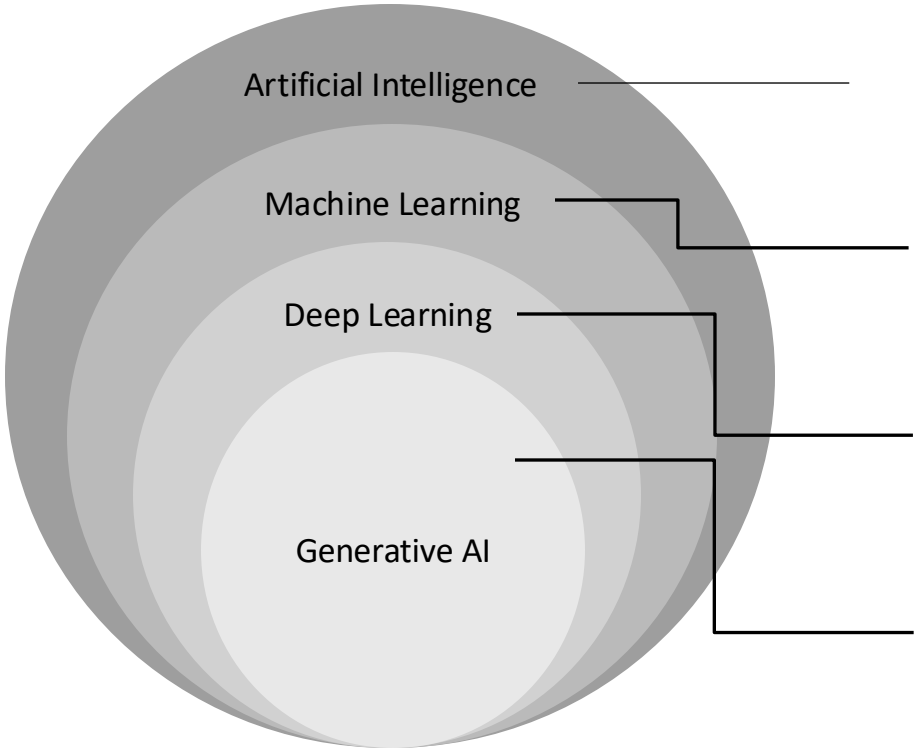
Use case	Model	Level	Attributes
Reservoir simulation	FNO, PINO	Advanced	Steady state, Multi-Node
Seismic wave	Fully Connected MLP PINN	Intermediate	Steady state, Multi-Node
Wave equation	Fully Connected MLP PINN	Intermediate	Steady state, Multi-Node

Healthcare

Use case	Model	Level	Attributes
Aneurysm modeling using STL geometry	Fully Connected MLP PINN	Intermediate	Steady state, Multi-Node

Deep Learning Overview

Where Generative AI Sits within Artificial Intelligence



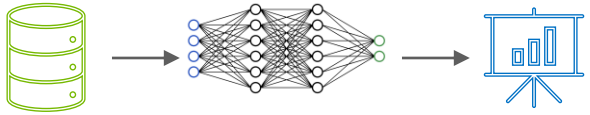
Solves a task by making computers to mimic human behavior

Learn → Analyze → Predict

Trains a machine to solve a specific AI problem

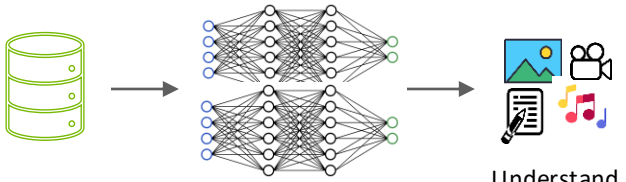
Data → Stats & Math Algorithms → Prediction

Uses neural networks to solve an AI problem



Data → Neural Networks → Prediction

Learns patterns and trends from the training data using neural networks, understand the context to create new content that mimics human generated content



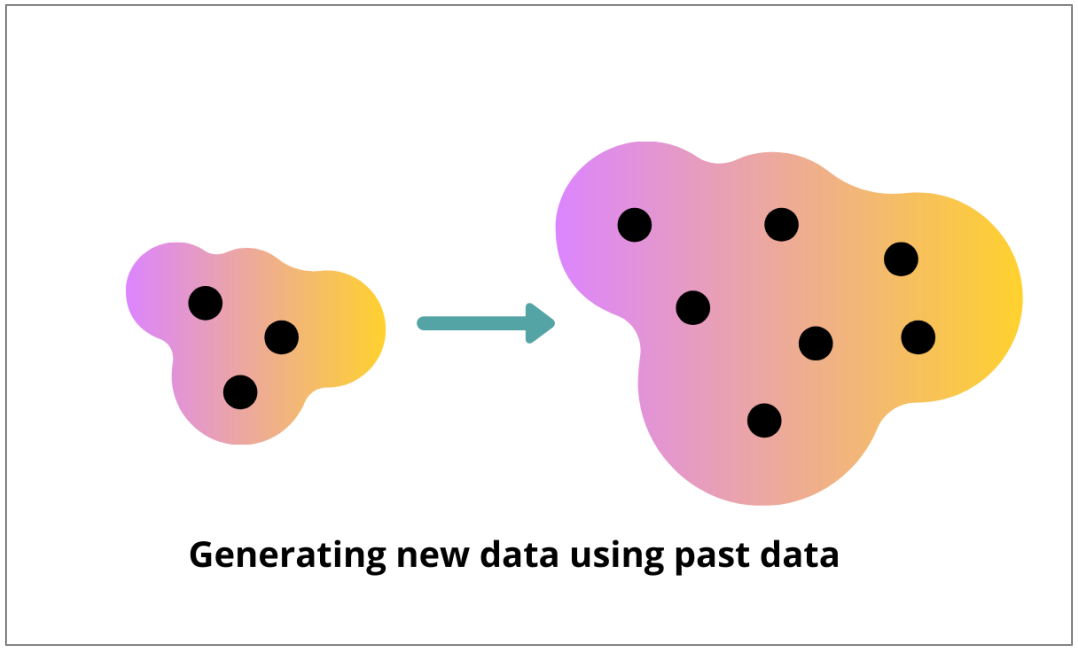
Training Data (a large dataset) → Neural Networks → Understand Context and Generate New and Unique Content

What is Generative AI?

Tips to Get Started

Generative AI refers to machine learning algorithms that enable computers to use existing or past content like text, audio and video files, images, and even code to generate new possible content.

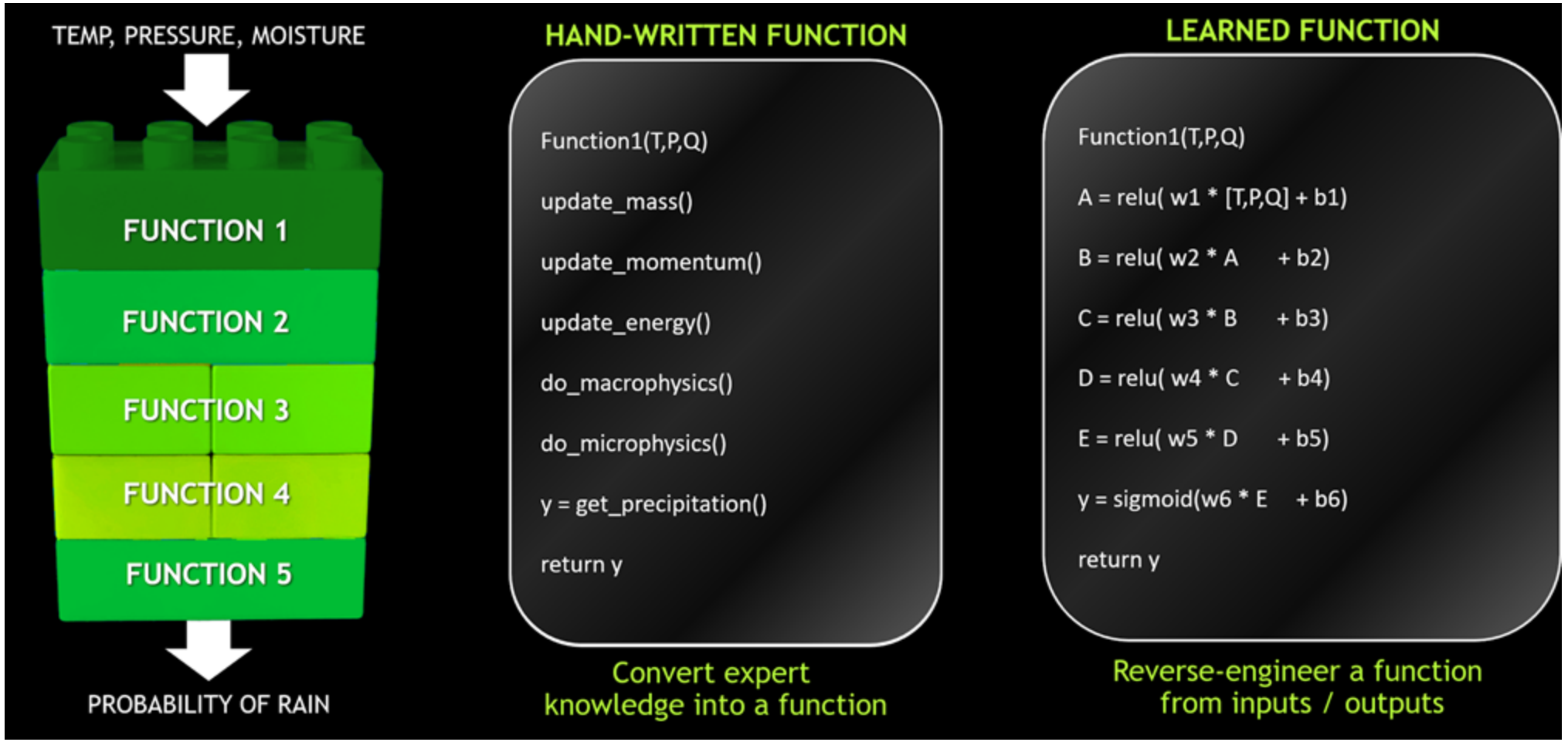
The main idea is to generate completely original artifacts that would look like the real deal.



Generating new data using past data

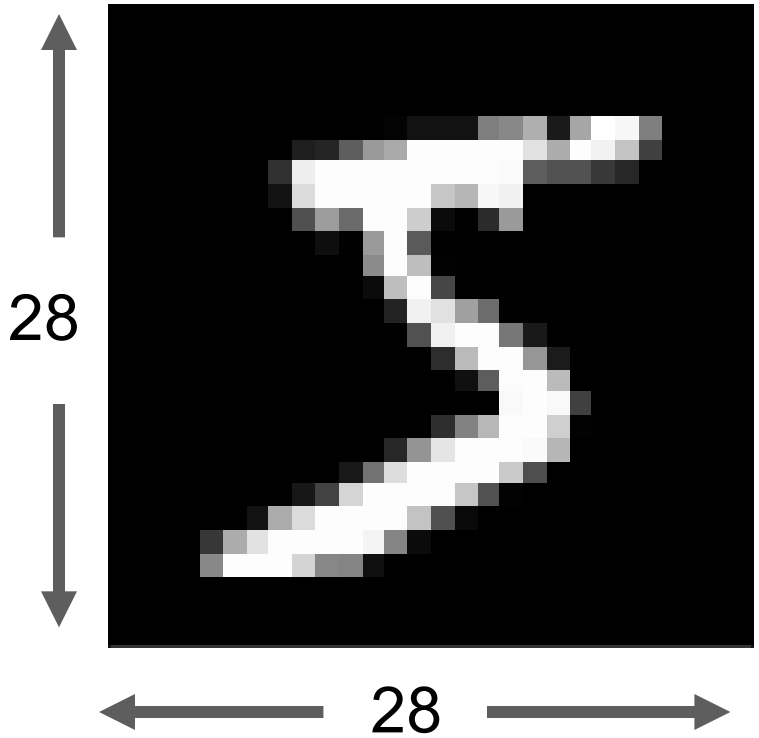
深度學習概述

AI = A SOFTWARE that write SOFTWARE



mnist 資料準備

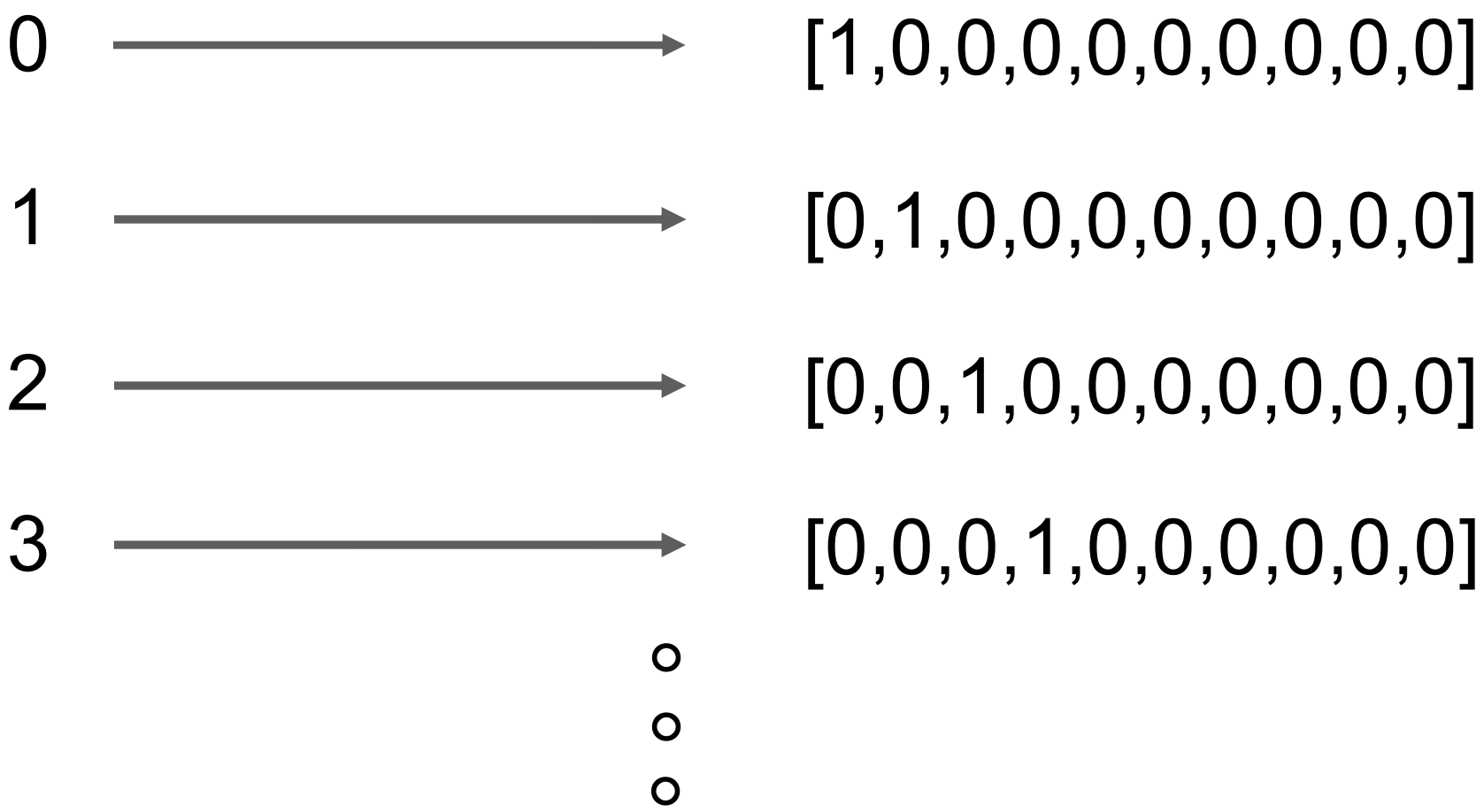
以陣列形式輸入



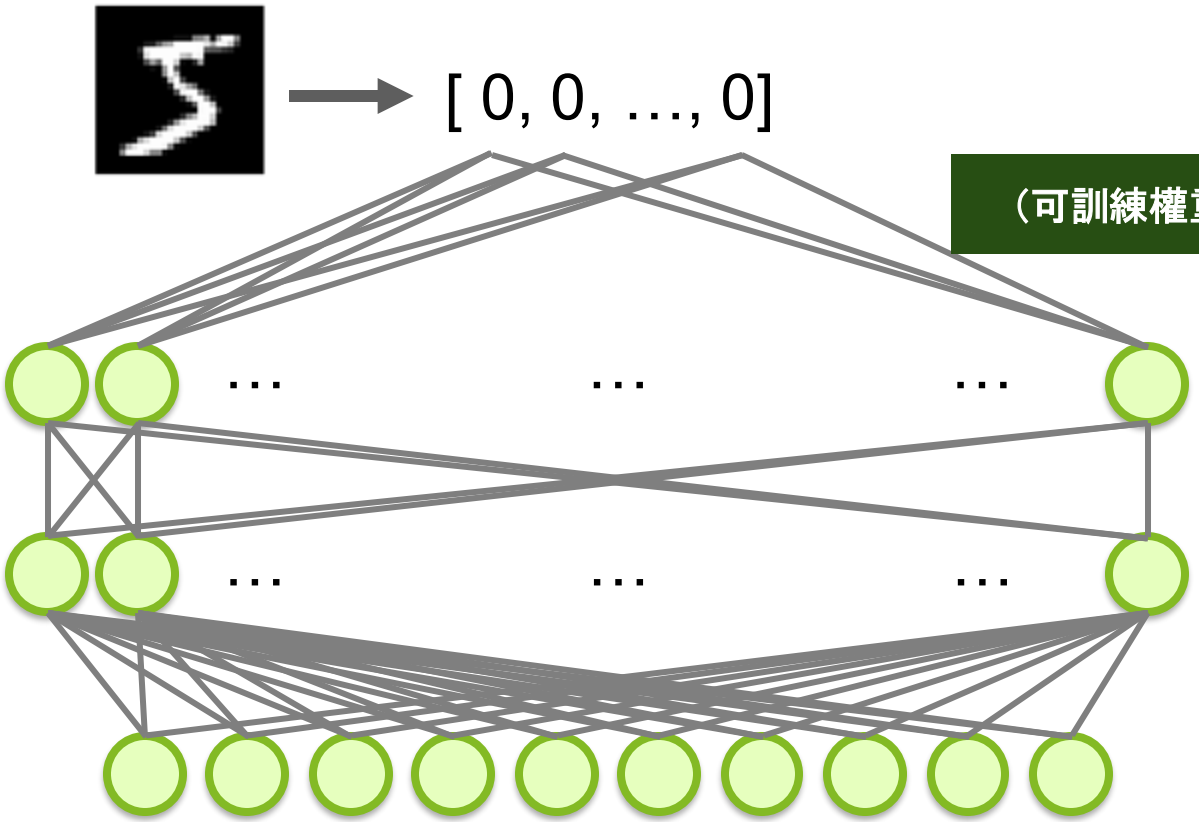
[0,0,0,24,75,184,185,78,32,55,0,0,0…]

mnist 資料準備

類別目標 （One-Hot Encoding）



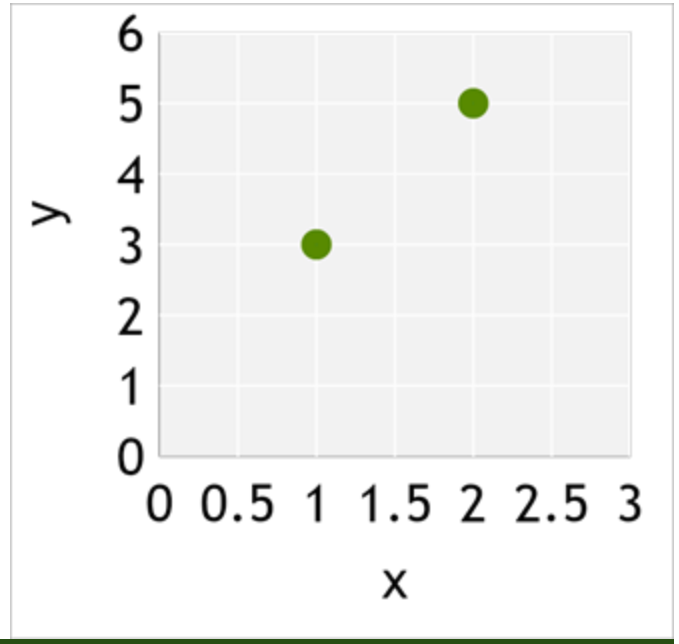
未經訓練的模型



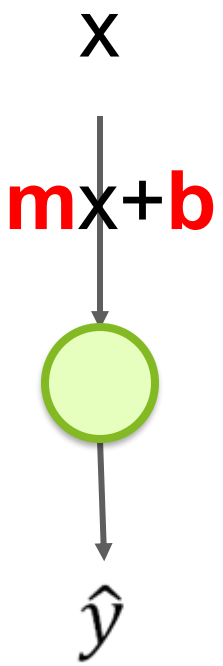
簡化的模型

$$y = mx + b$$

x	y
1	3
2	5



觀測資料



$m = ?$

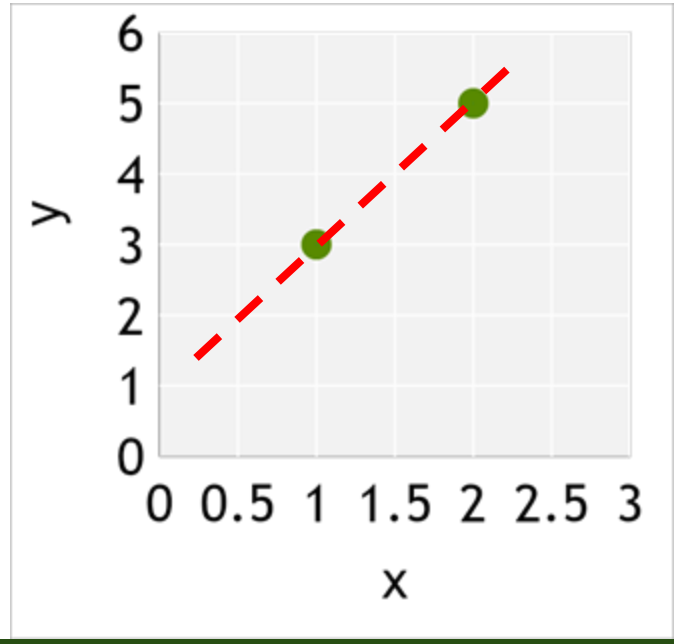
$b = ?$

類神經網路 （簡化）

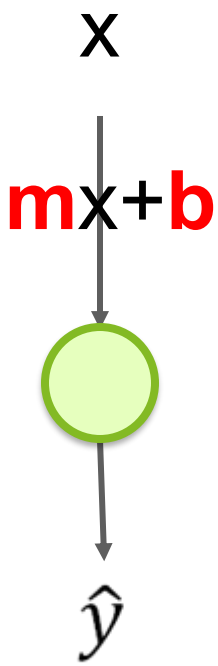
簡化的模型

$$y = mx + b$$

x	y
1	3
2	5



觀測資料



$m = ?$

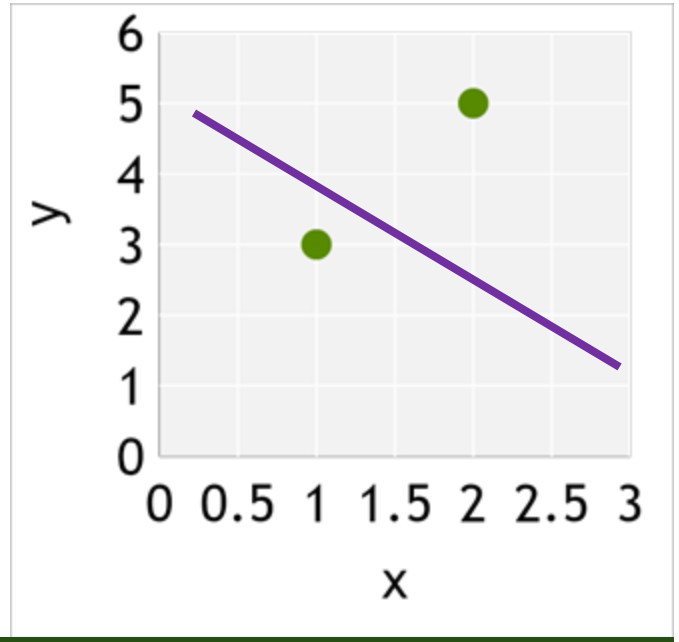
$b = ?$

類神經網路 （簡化）

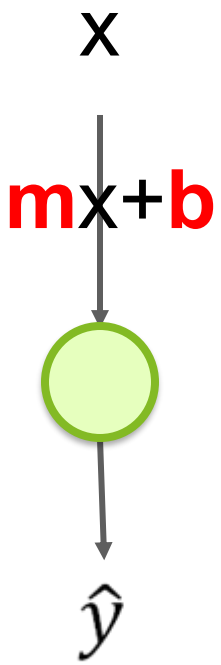
簡化的模型

$$y = mx + b$$

x	y	y^
1	3	4
2	5	3



觀測資料



類神經網路 （簡化）

隨機初始

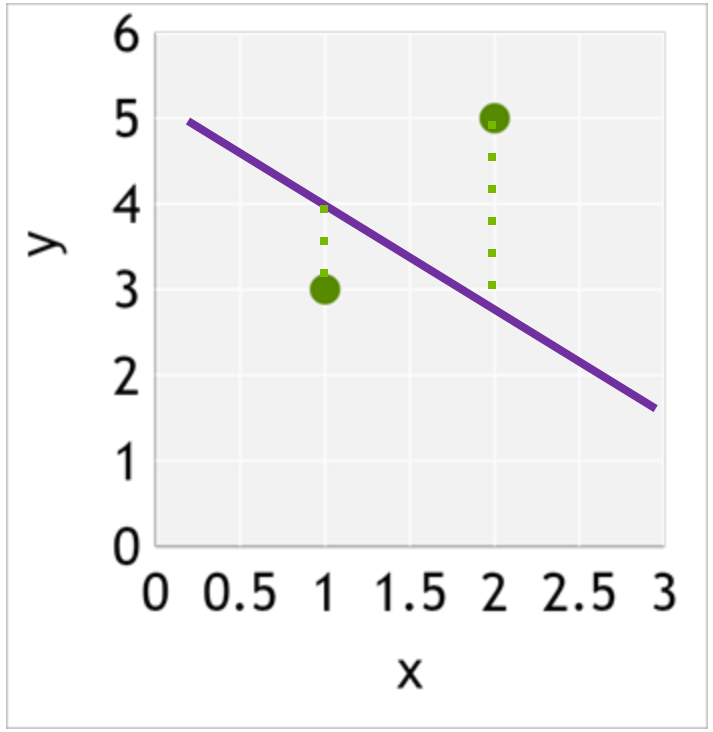
$$m = -1$$

$$b = 5$$

簡化的模型

$$y = mx + b$$

x	y	y^	err^2
1	3	4	1
2	5	3	4
		MSE =	2.5
		RMSE =	1.6

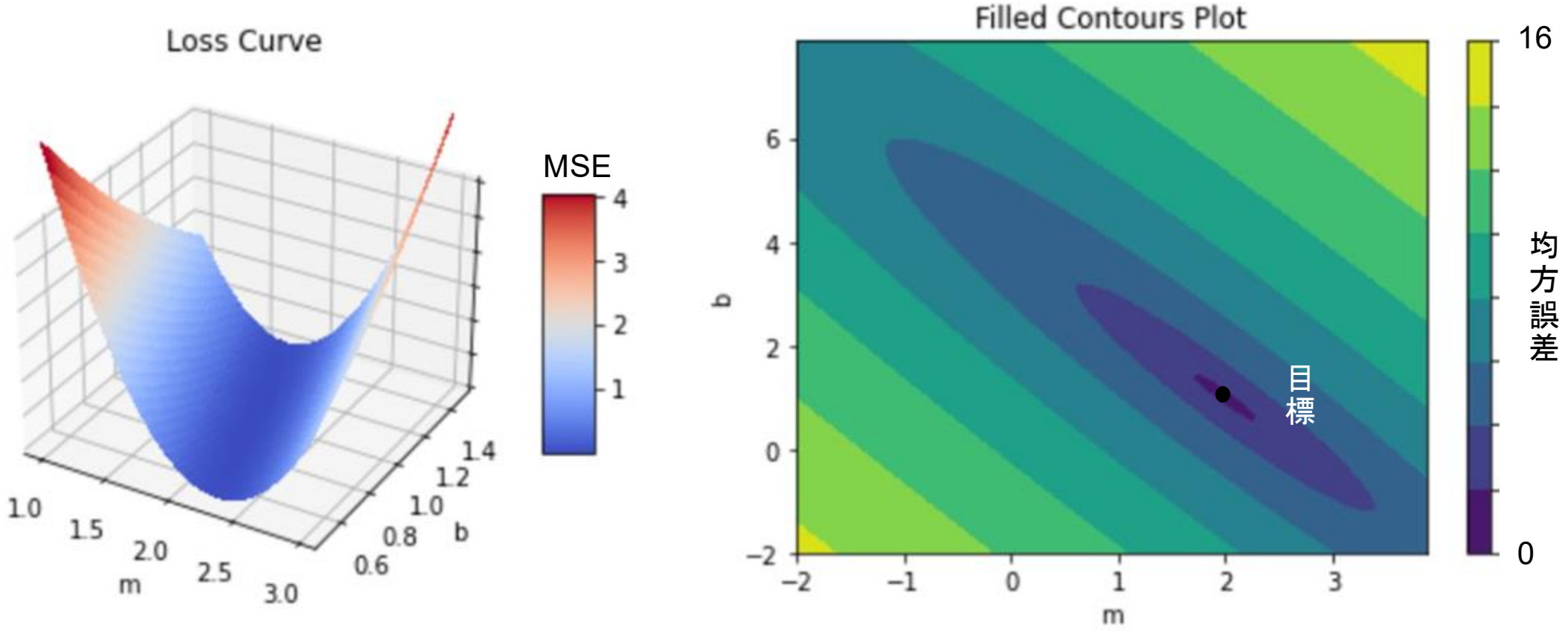


Mean Square Error（最小平方誤差）

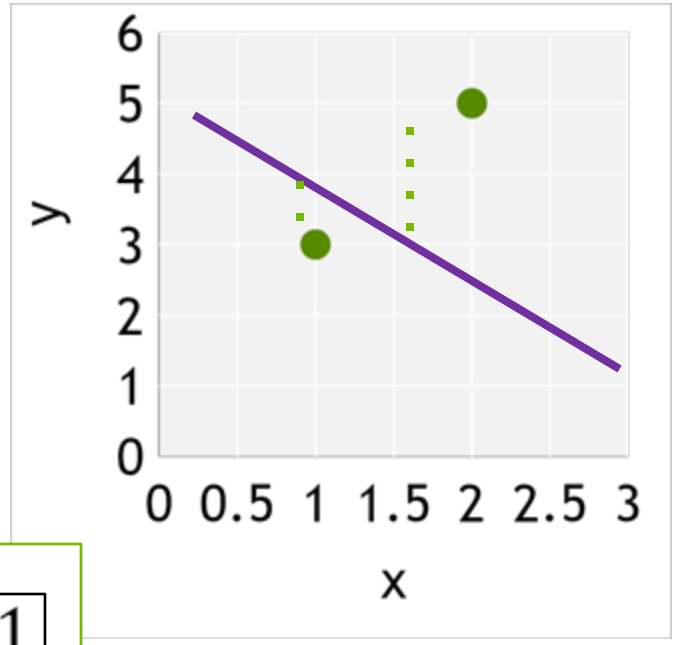
$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y - \hat{y})^2$$

$$RMSE = \frac{1}{n}\sqrt{\sum_{i=1}^{n}(y - \hat{y})^2}$$

損失曲線 (Loss Landscape)

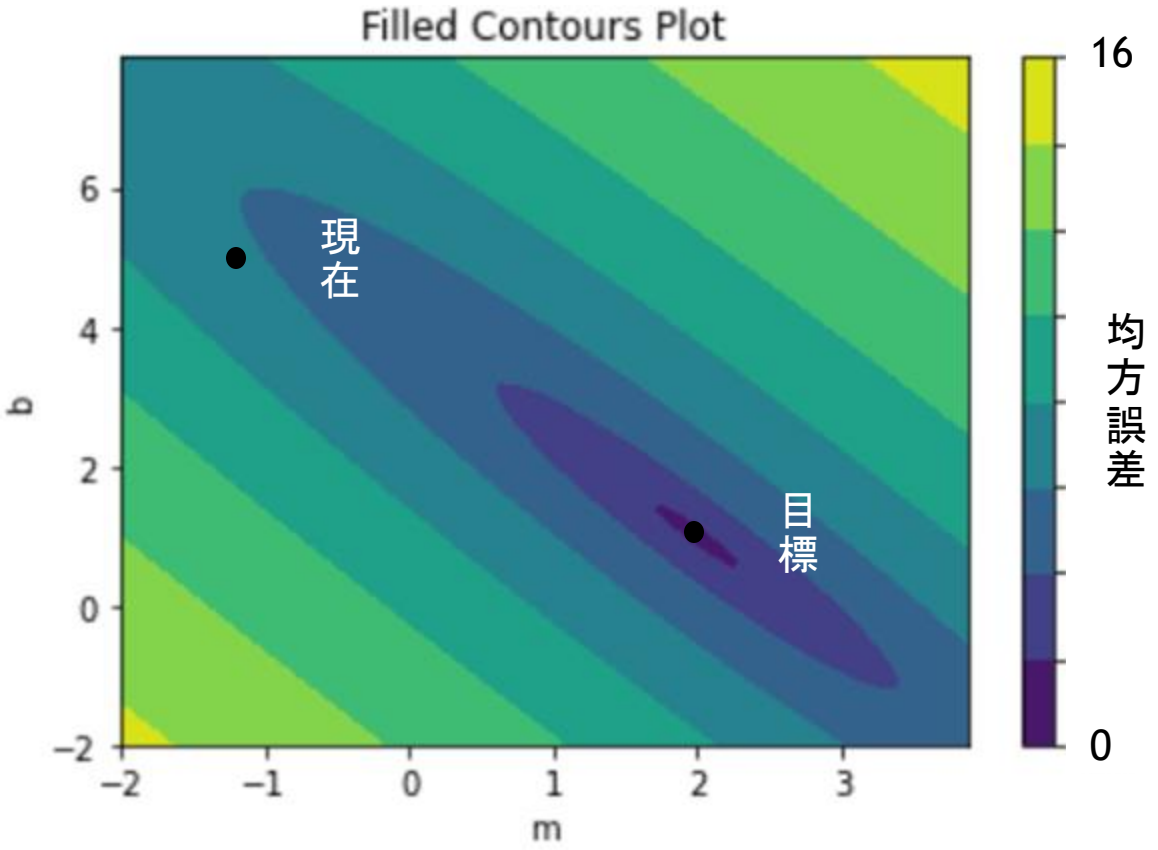


損失曲線 (Loss Landscape)



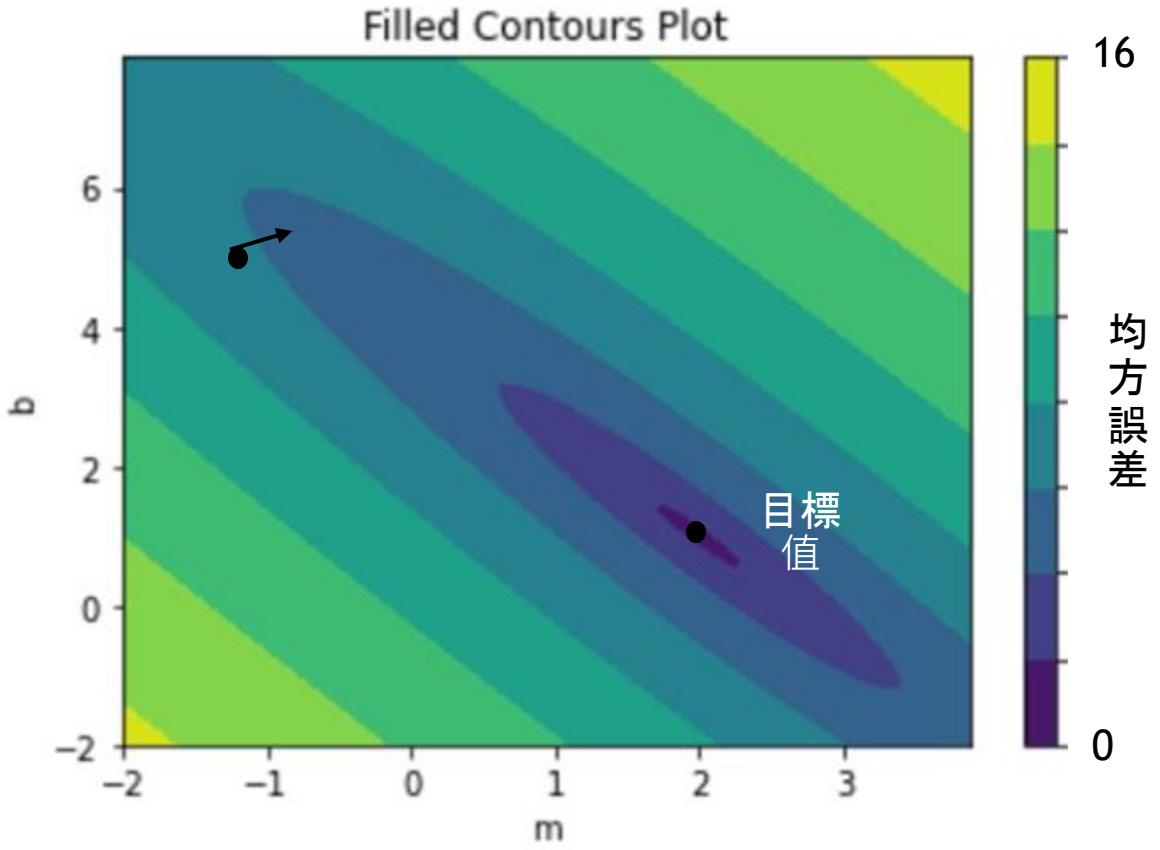
$m = -1$

$b = 5$



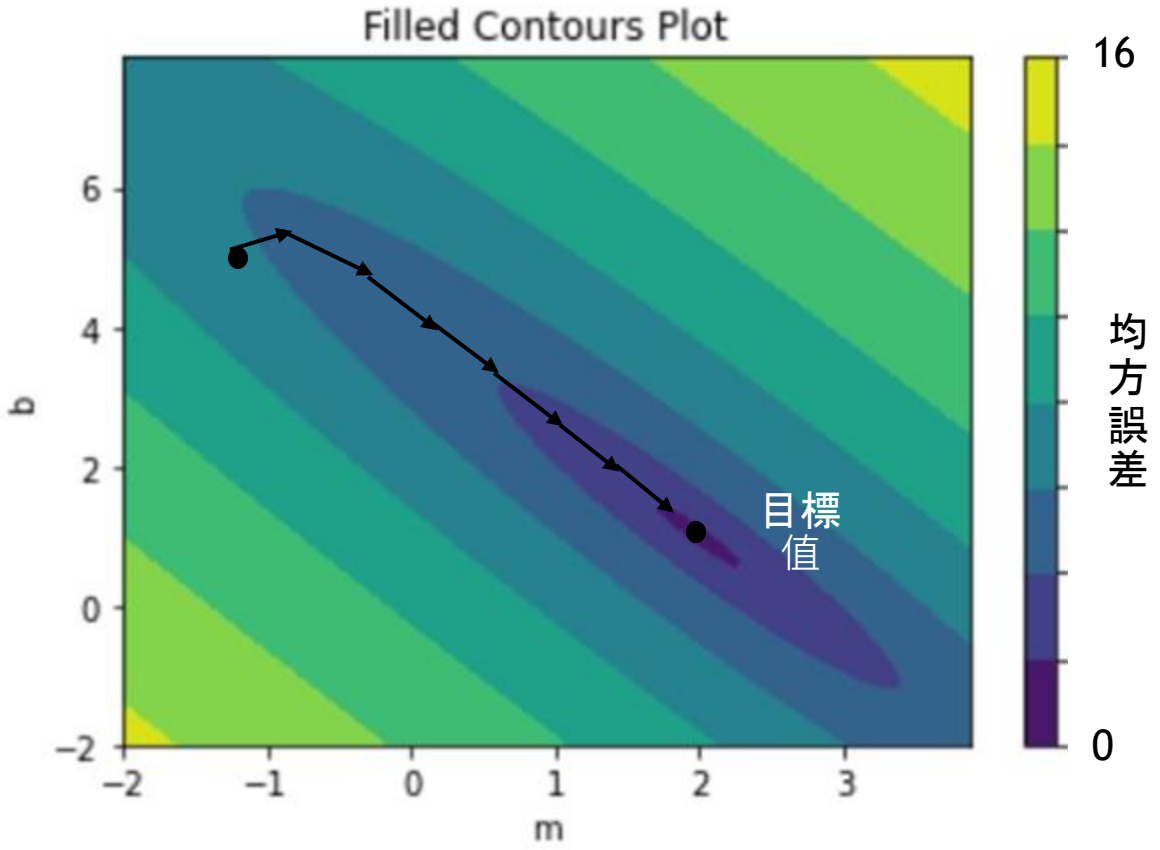
損失曲線 (Loss Landscape)

週期	(Epoch) 以完整資料集進行模型更新
批次	(Batch) 完整資料集的範例
梯度	(Gradient) 哪個方向的損失率降低最多
學習率	(Learning Rate) 移動距離
步驟	(Step) 更新權重參數

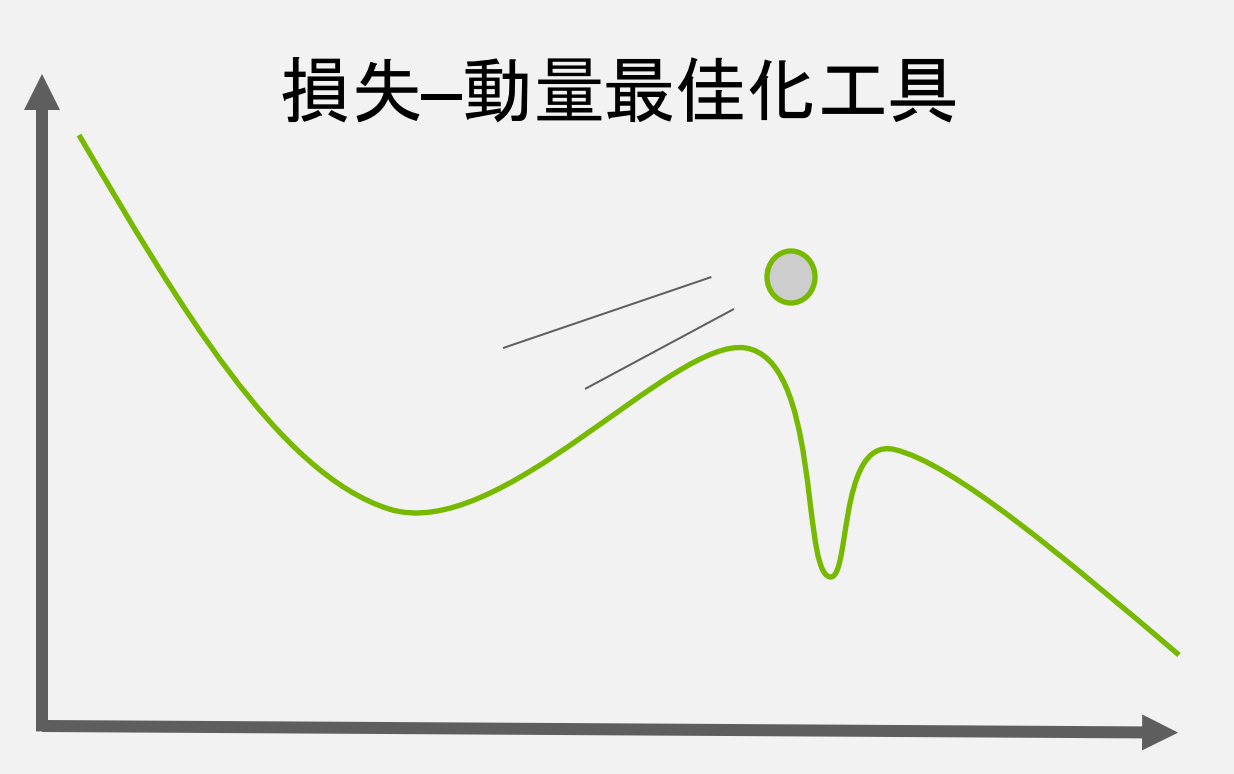


損失曲線 (Loss Landscape)

週期	(Epoch) 以完整資料集進行模型更新
批次	(Batch) 完整資料集的範例
梯度	(Gradient) 哪個方向的損失率降低最多
學習率	(Learning Rate) 移動距離
步驟	(Step) 更新權重參數

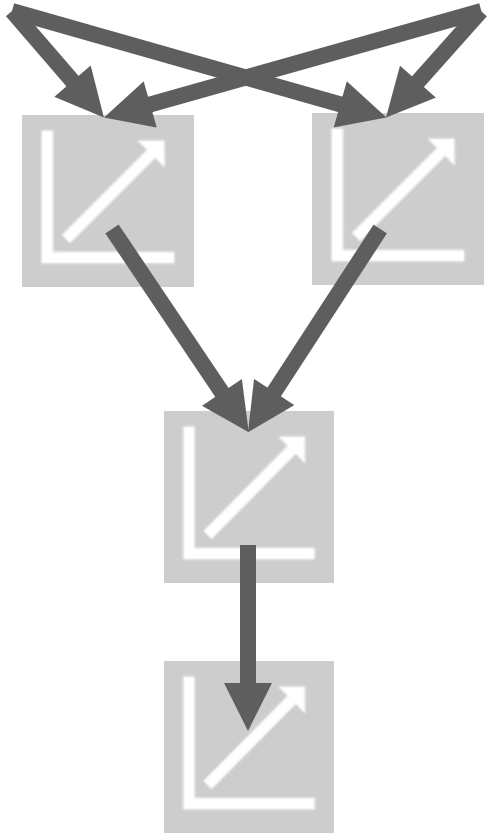


優化器 (Optimizer)



- Adam
- Adagrad
- RMSprop
- SGD

建立網路 (Network)

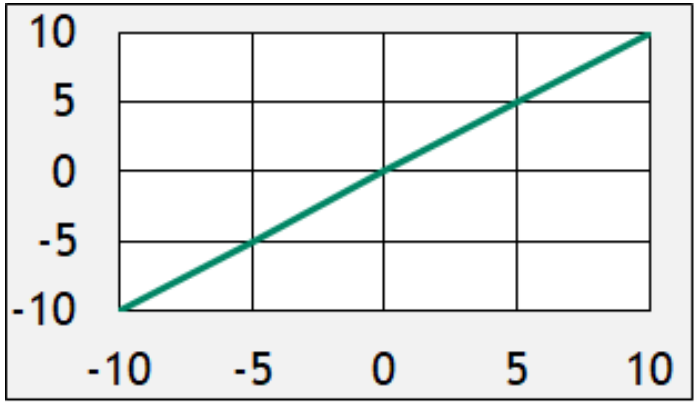


- 擴充為更多輸入值
- 可以連結神經元
- 如果所有迴歸都是線性的，則輸出也會是線性迴歸

激活函數 (Activation Function)

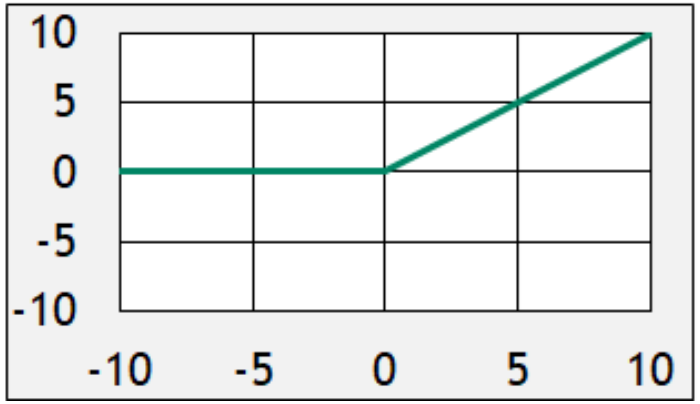
線性

```
# Multiply each input
# with a weight (w) and
# add intercept (b)
y_hat = wx+b
```



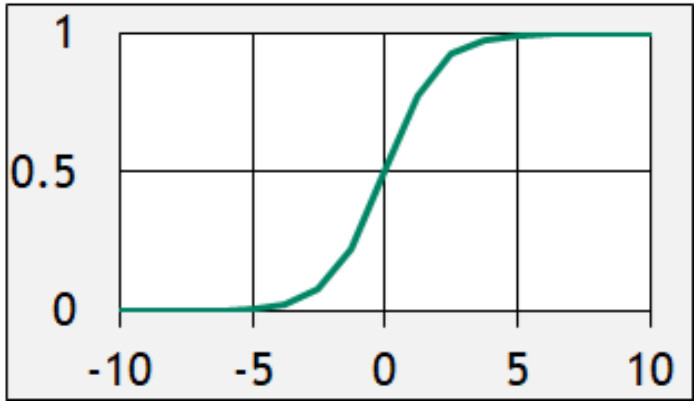
ReLU

```
# Only return result
# if total is positive
linear = wx+b
y_hat = linear * (linear > 0)
```



Sigmoid

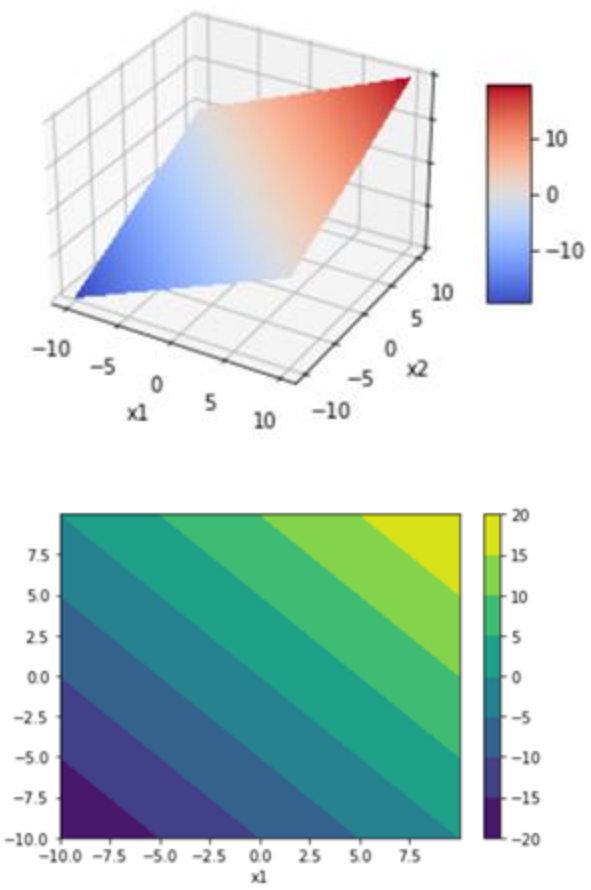
```
# Start with line
linear = wx + b
# Warp to - inf to 0
inf_to_zero = np.exp(-1 * linear)
# Squish to -1 to 1
y_hat = 1 / (1 + inf_to_zero)
```



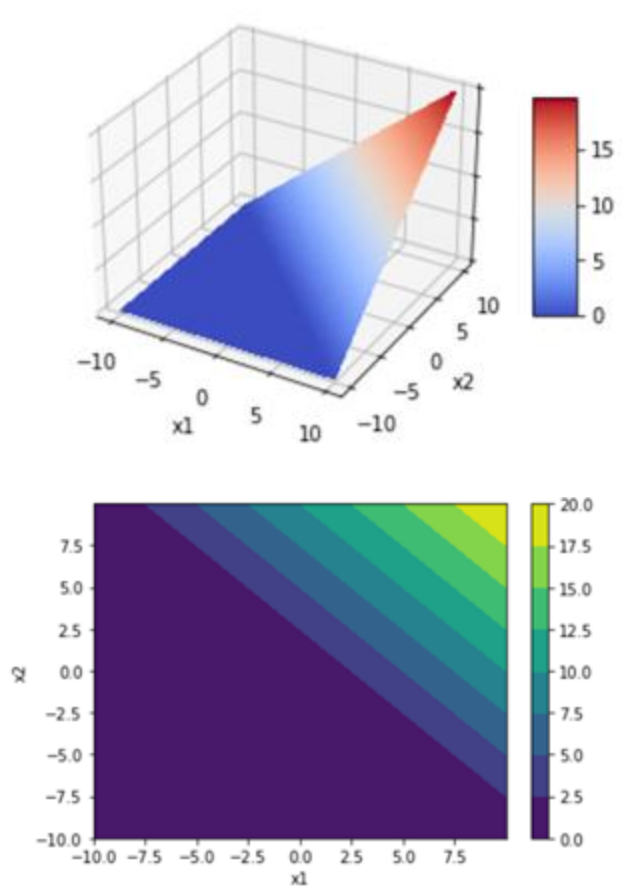
（Logistic Regression）

激活函數 (Activation Function)

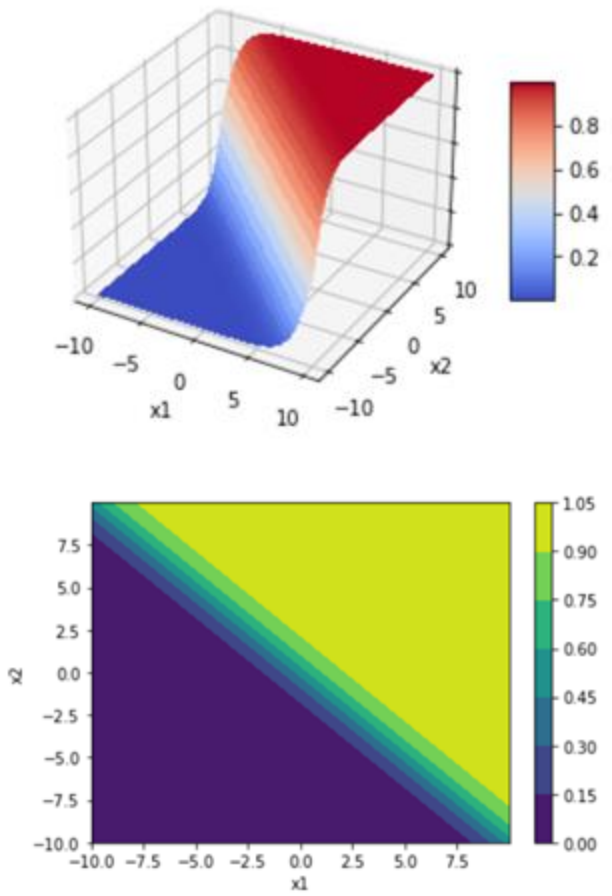
Linear



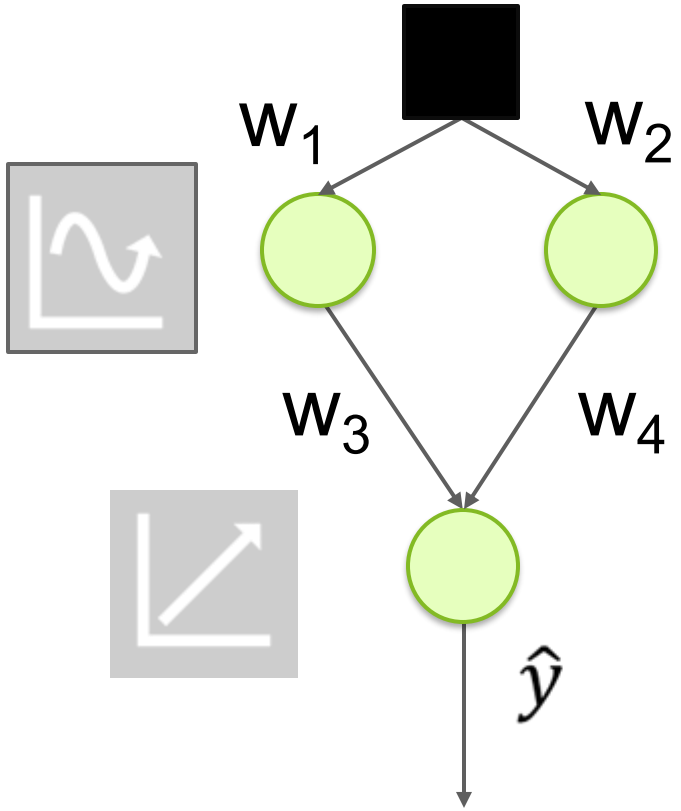
ReLU

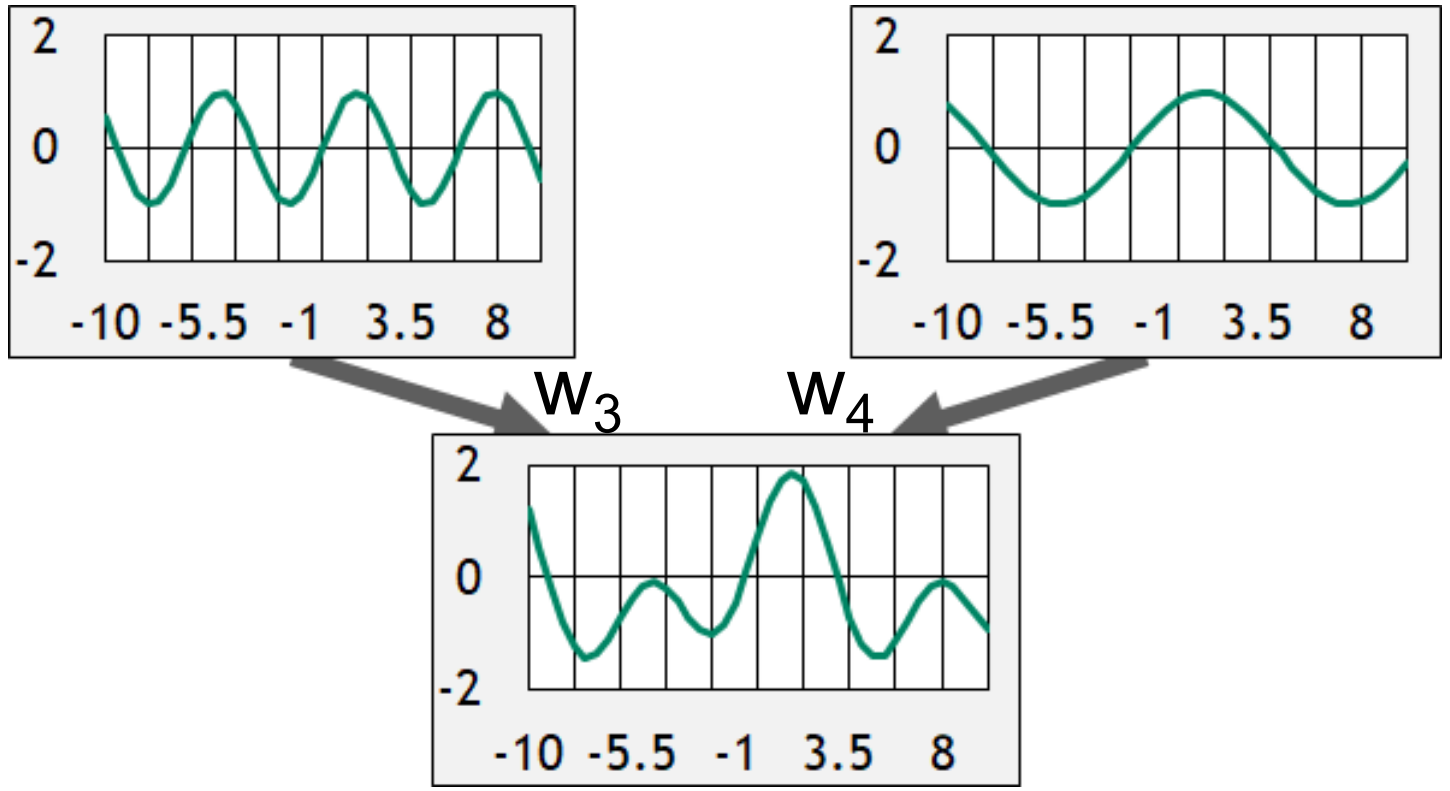


Sigmoid



激活函數 (Activation Function)

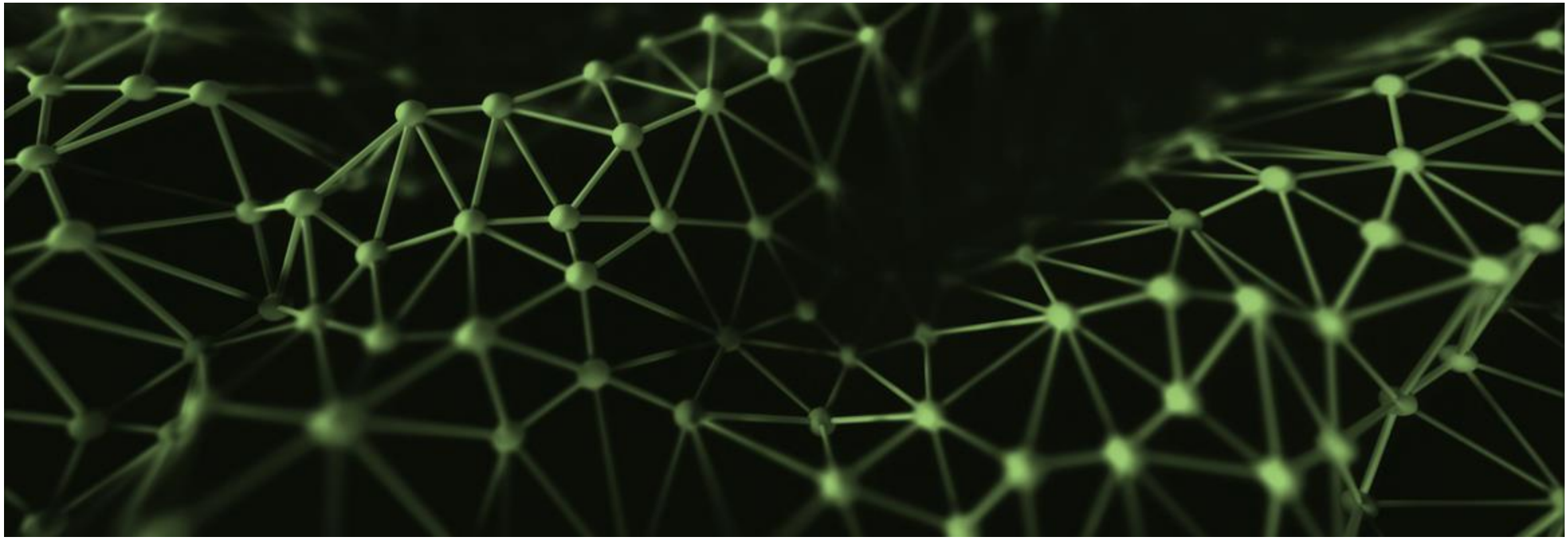




Universal approximation theorem

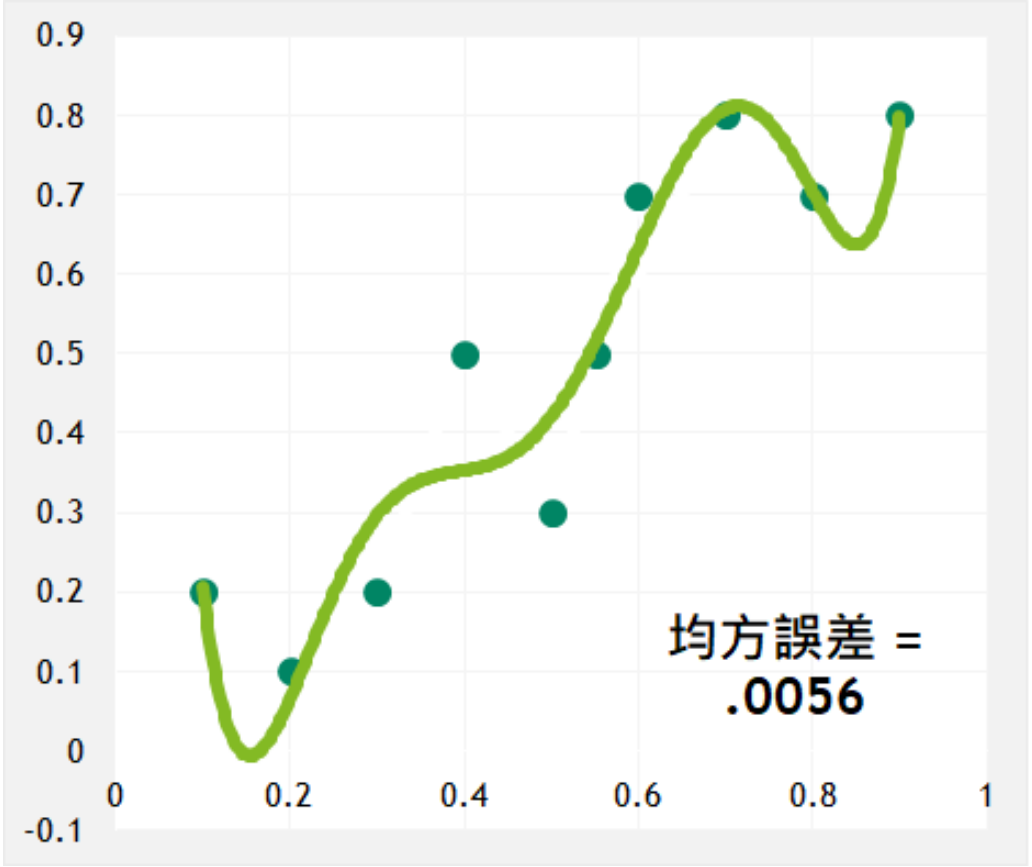
過度擬合 (Overfitting)

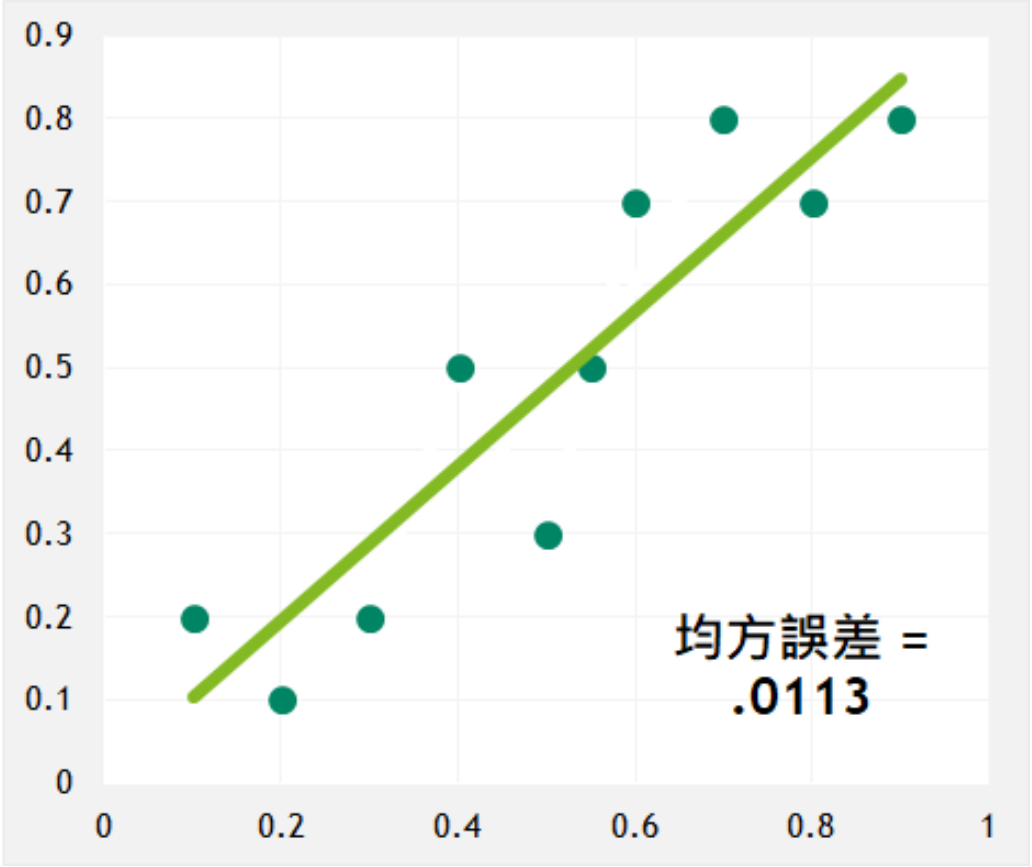
為何不建構一個超大型神經網路？



過度擬合 (Overfitting)

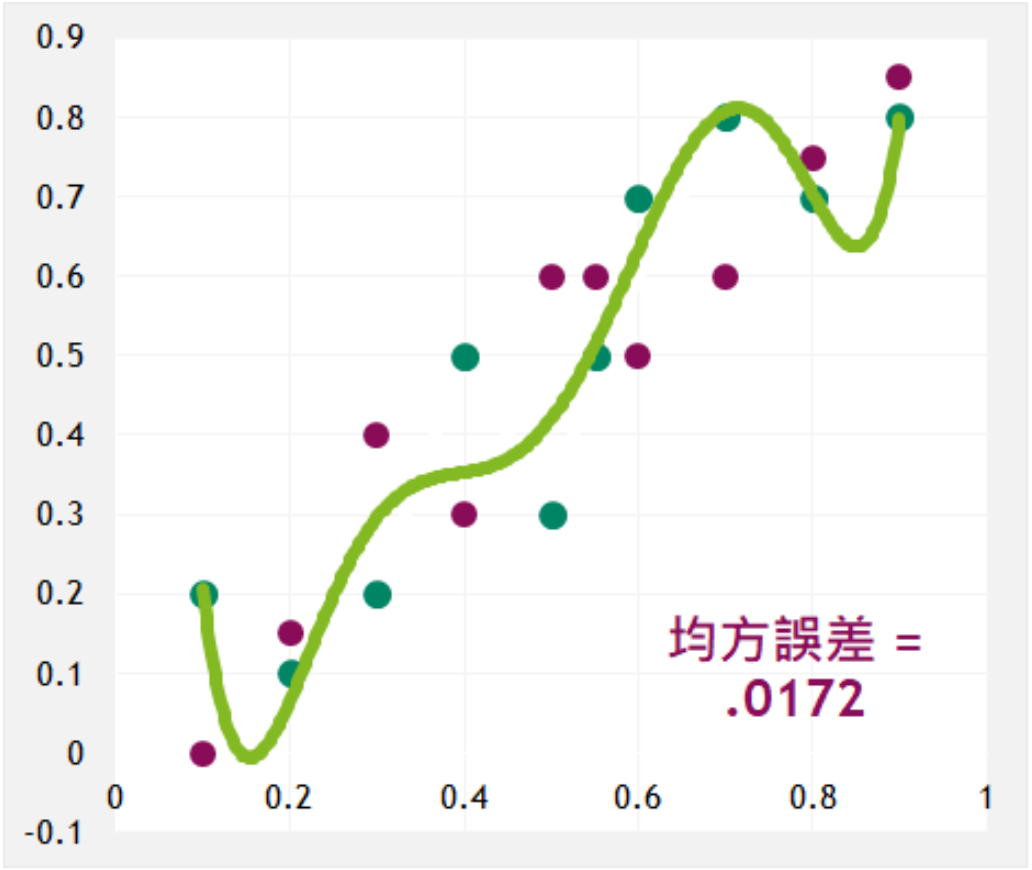
哪條趨勢線更理想？

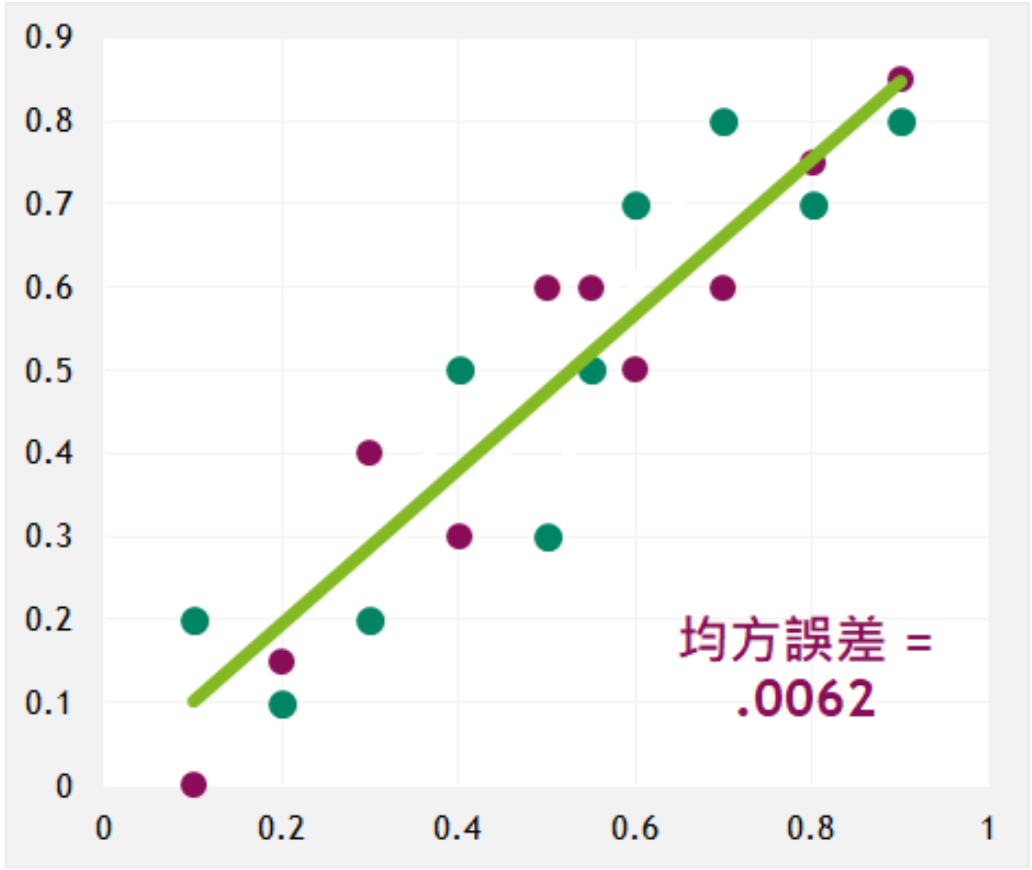




過度擬合 (Overfitting)

哪條趨勢線更理想？





訓練 VS 驗證資料

避免背誦

訓練資料 (Training Dataset)

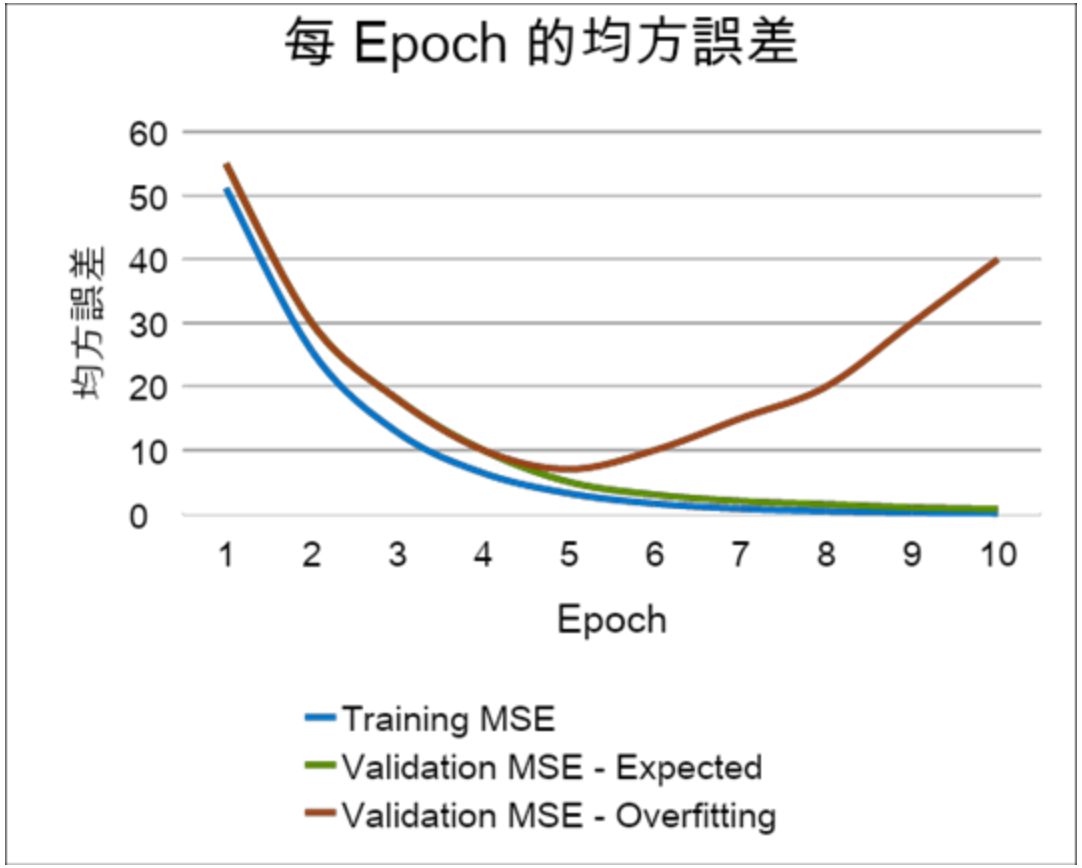
- 讓模型進行學習的核心資料集

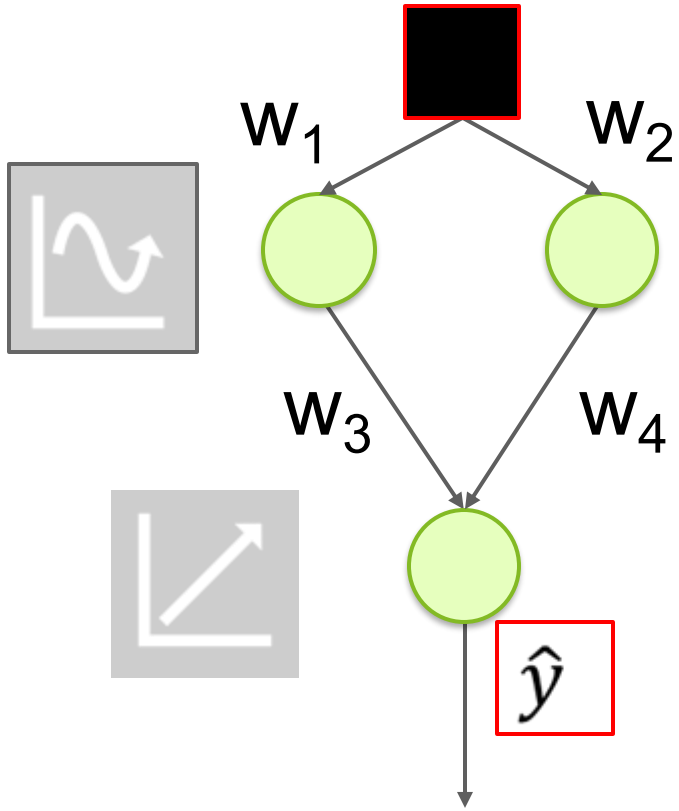
驗證資料 (Validation Dataset)

- 用來測試模型是否真的瞭解規則的新資料 (可歸納)

過度擬合 (Overfitting)

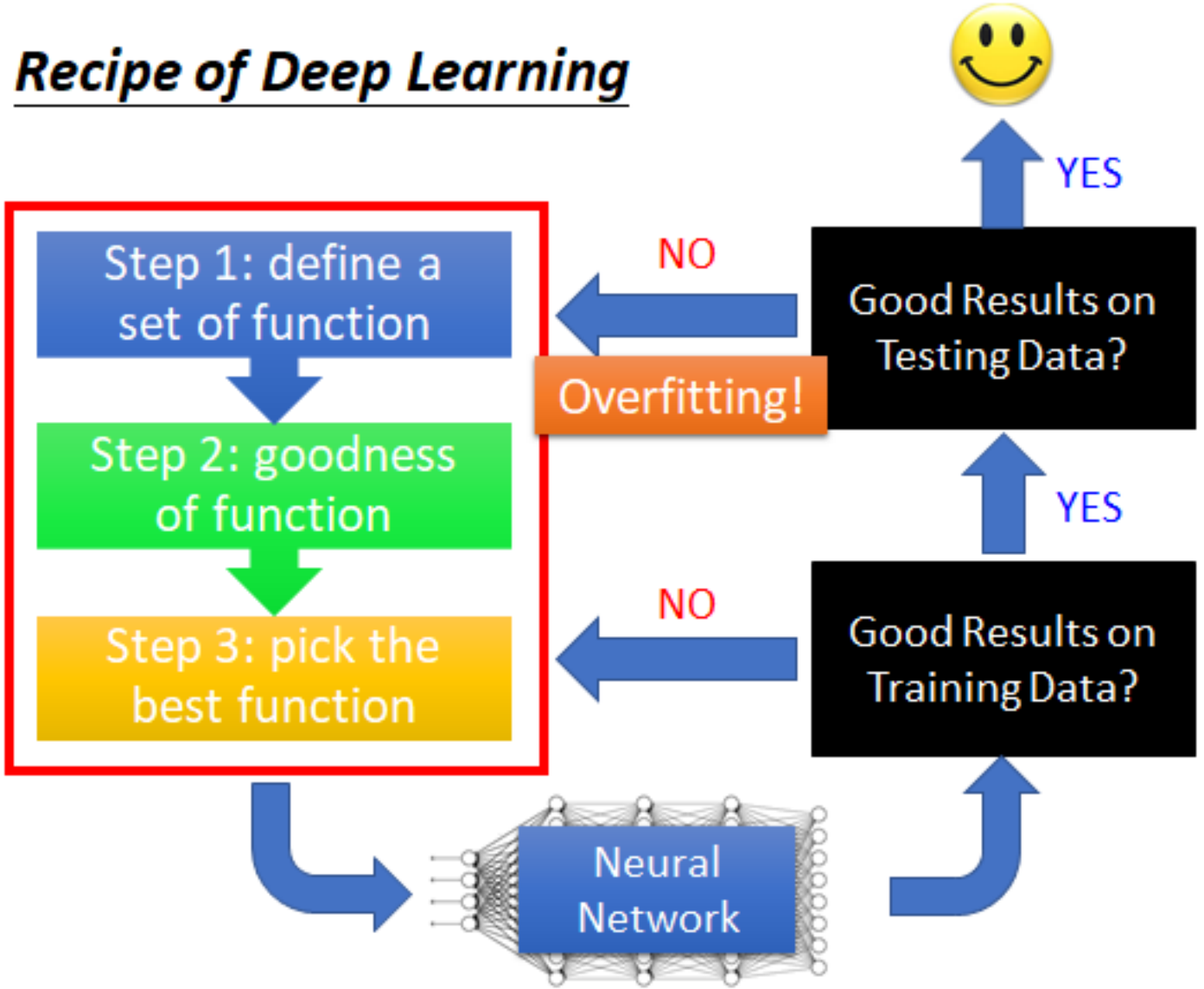
- 模型在訓練資料上效果很好，在驗證資料上卻效果不佳 (背誦的證據)
- 理想情況下，兩個資料集的精確度和損失率應該趨於接近





$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y - \hat{y})^2$$

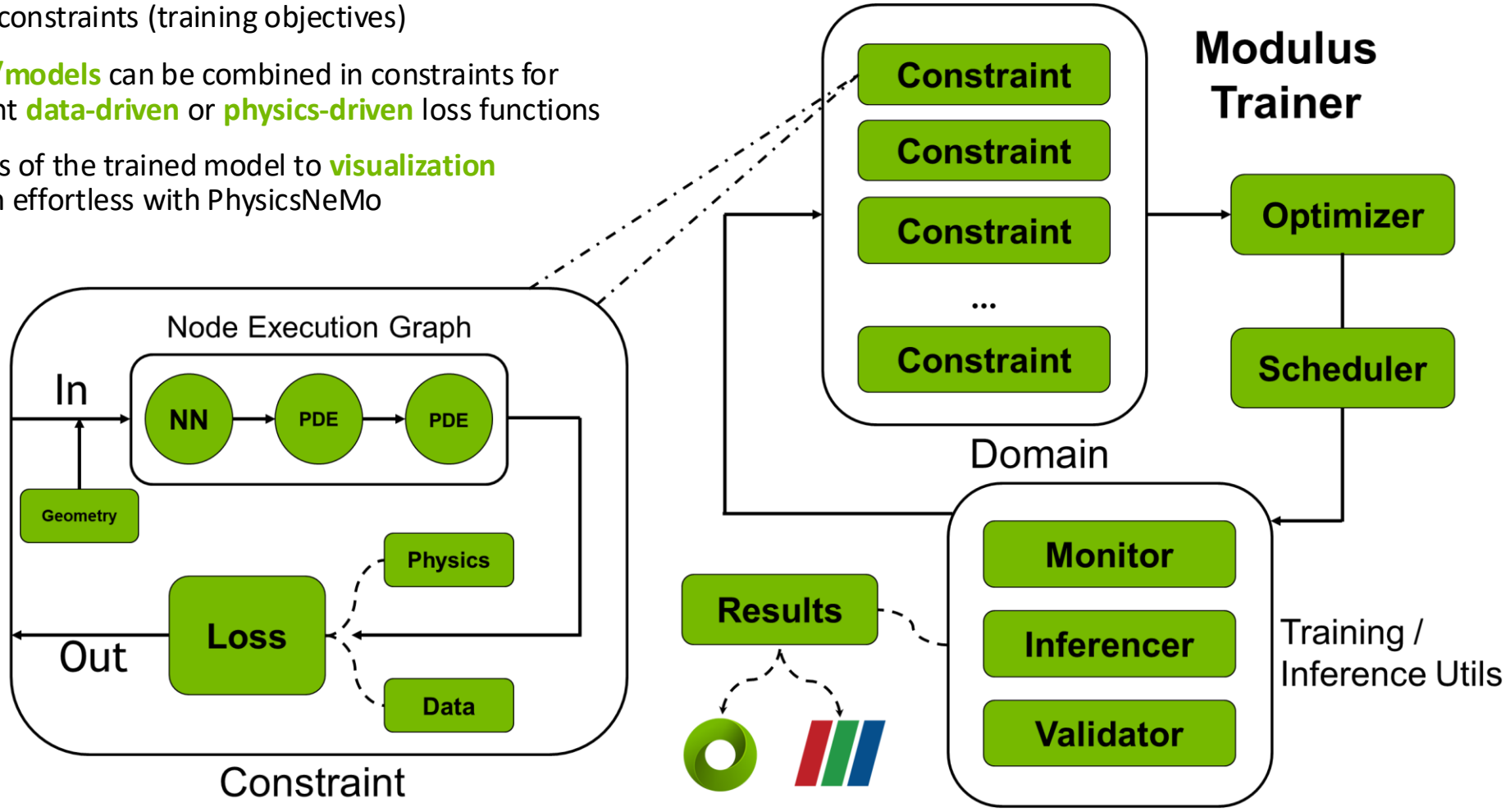
Recipe of Deep Learning



PhysicsNeMo Architecture, Traning

PhysicsNeMo Training

- PhysicsNeMo allows for **complex problems** to be described through sets of constraints (training objectives)
- Multiple **nodes/models** can be combined in constraints for learning different **data-driven** or **physics-driven** loss functions
- Exporting results of the trained model to **visualization software** is then effortless with PhysicsNeMo



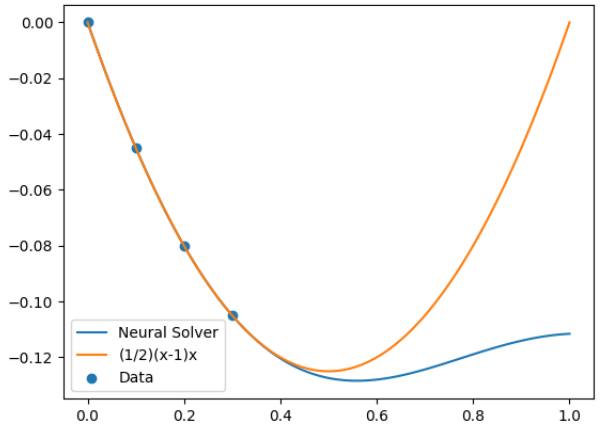
Physics Informed Neural Networks (PINNs) in PhysicsNeMo

A problem with NNs and the promise of PINNs

Data Only

$$L_{data} = \sum_{x_i \in data} (u_{net}(x_i) - u_{true}(x_i))^2$$

$$L_{total} = L_{data}$$

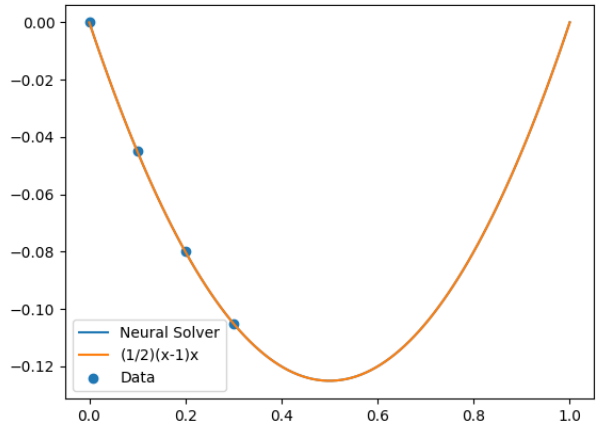


Data + Physics

$$\frac{\delta^2 u}{\delta x^2}(x) = 1$$

$$L_{physics} = \sum_{x_j \in domain} \left(\frac{\delta^2 u_{net}}{\delta x^2}(x_i) - f(x_i) \right)^2$$

$$L_{total} = L_{data} + L_{physics}$$

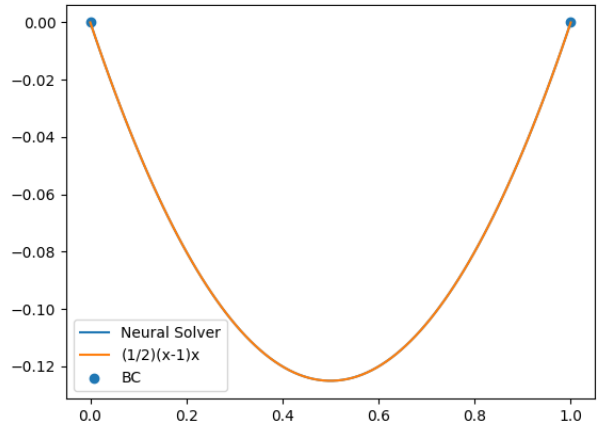


Physics only

$$\frac{\delta^2 u}{\delta x^2}(x) = 1 \qquad u(0) = u(1) = 0$$

$$L_{physics} = L_{residual} + L_{BC}$$

$$L_{total} = L_{physics}$$



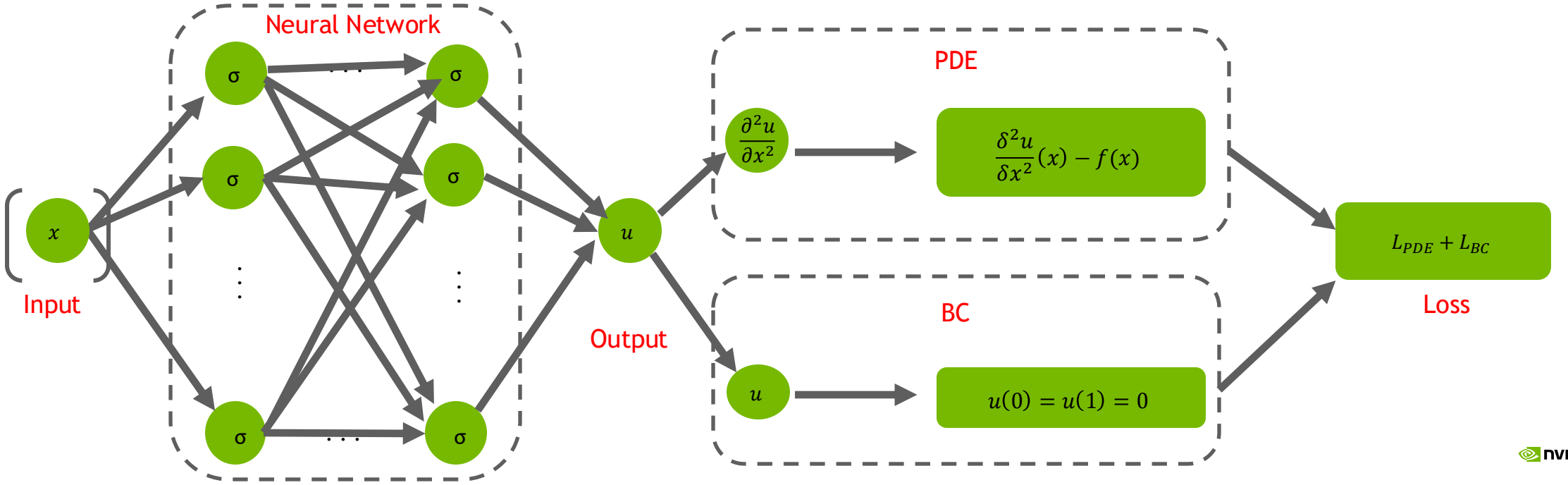
PINNs Theory: Neural Network Solver

- Goal: Train a neural network to satisfy the boundary conditions and differential equations by constructing an appropriate loss function
  - Consider an example problem:

$$\mathbf{P}: \begin{cases} \frac{\delta^2 u_{net}}{\delta x^2}(x) = f(x) \\ u_{net}(0) = u_{net}(1) = 0 \end{cases}$$

Forward Problem

- We construct a neural network $\boldsymbol{u_{net}(x)}$ which has a single value input $x \in \mathbb{R}$ and single value output $u_{net}(x) \in \mathbb{R}$.
- We assume the neural network is **infinitely differentiable** $u_{net} \in C^{\infty}$ - Use activation functions that are infinitely differentiable



PINNs Theory: Neural Network Solver

Loss formulation

- Construct the loss function. We can compute the second order derivatives $\left(\frac{\delta^2 u_{net}}{\delta x^2}(x)\right)$ using Automatic differentiation

$$L_{BC} = u_{net}(0)^2 + u_{net}(1)^2$$

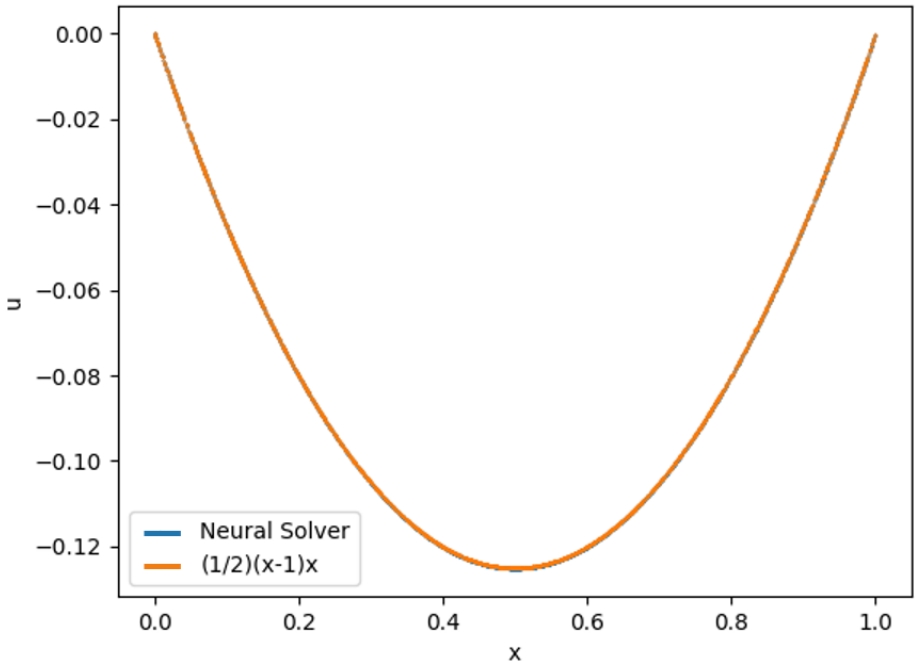
$$L_{Residual} = \int_0^1 \left(\frac{\delta^2 u_{net}}{\delta x^2}(x) - f(x)\right)^2 dx \approx \left(\int_0^1 dx\right)\frac{1}{N}\sum_{i=0}^{N}\left(\frac{\delta^2 u_{net}}{\delta x^2}(x_i) - f(x_i)\right)^2$$

- Where x_i are a batch of points in the interior $x_i \in (0, 1)$. Total loss becomes $L = L_{BC} + L_{Residual}$
- Minimize the loss using optimizers like Adam

PINNs Theory: Neural Network Solver

Results

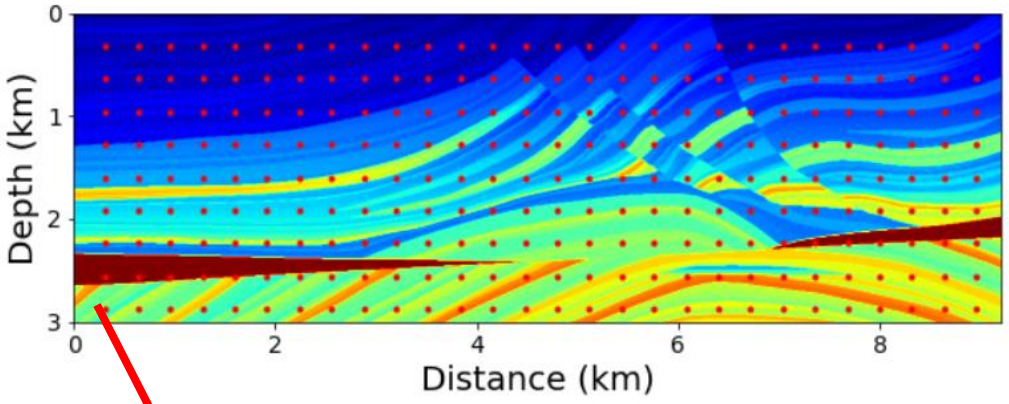
- For $f(x) = 1$, the true solution is $\frac{1}{2}(x-1)x$. After sufficient training we have,



Comparison of the solution predicted by Neural Network with the analytical solution

AN EXAMPLE: SEISMIC PROBLEMS

Point cloud representation (not a mesh)



PHYSICS

Neural Network

Input

Output

PDE

$\frac{\partial^2 u}{\partial t^2} - c^2\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial z^2}\right)$

IC

$u(x,z,t) - u_D$

BC

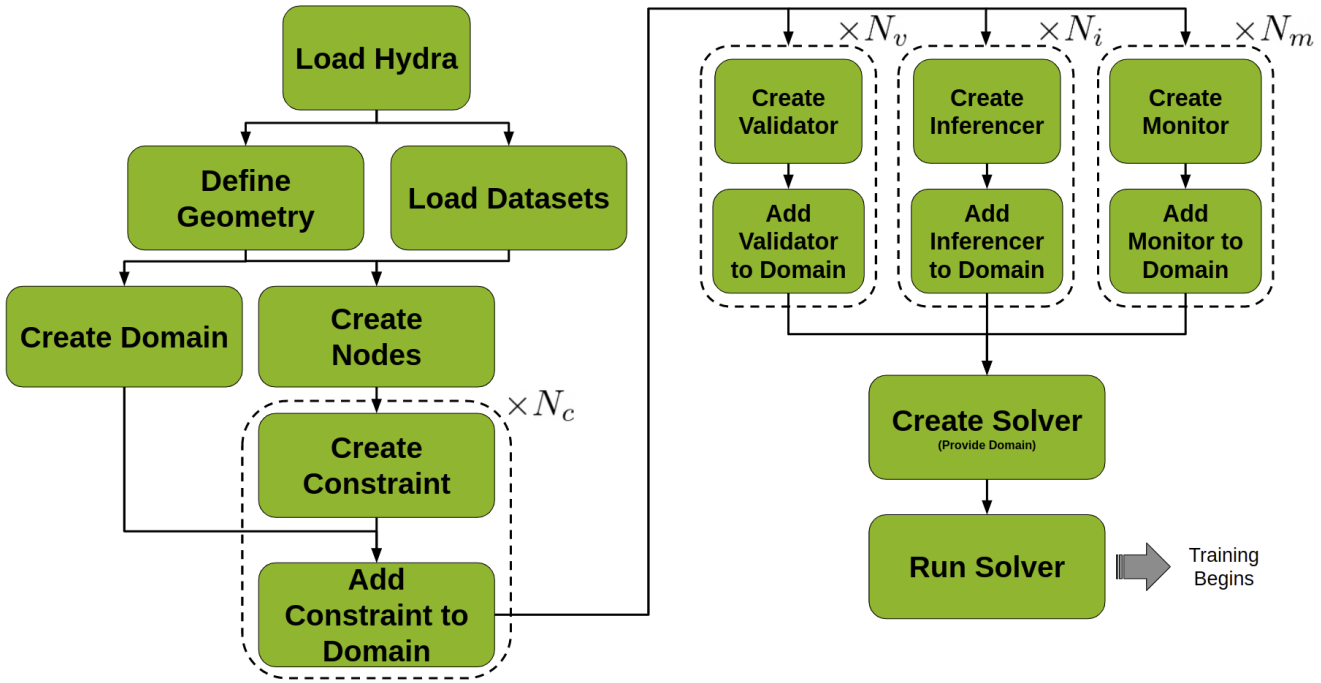
$\frac{\partial u(x,z,t)}{\partial n} - u_N$

$L_{PDE} + L_{IC} + L_{BC}$

Loss

PhysicsNeMo: Anatomy of a project

Overview

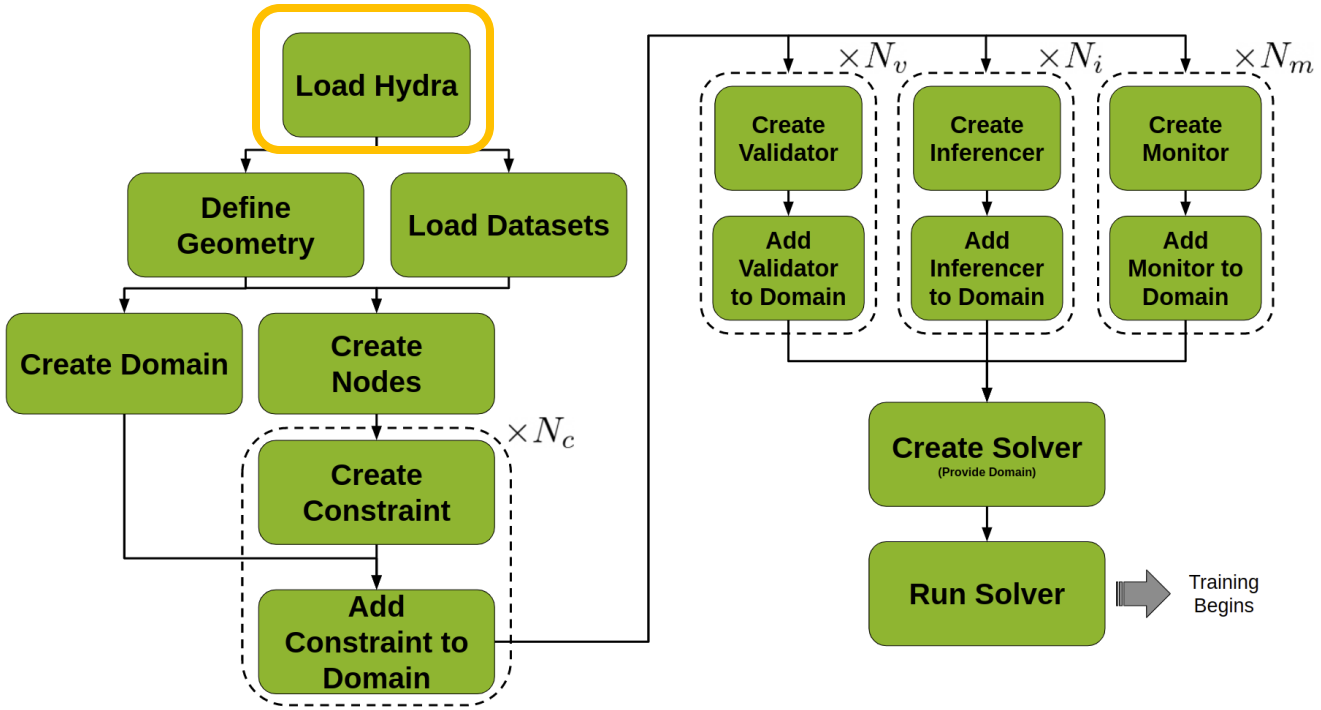


PhysicsNeMo works by:

- Writing models which include at least one adaptable function (a NN)
- Writing objective functions as a combination of these models
- Describing the geometry/dataset where the models should be evaluated
- Minimizing the objective functions by using the provided data, by sampling the geometry, or both
- Running the models to obtain the desired effect

PhysicsNeMo: Anatomy of a project

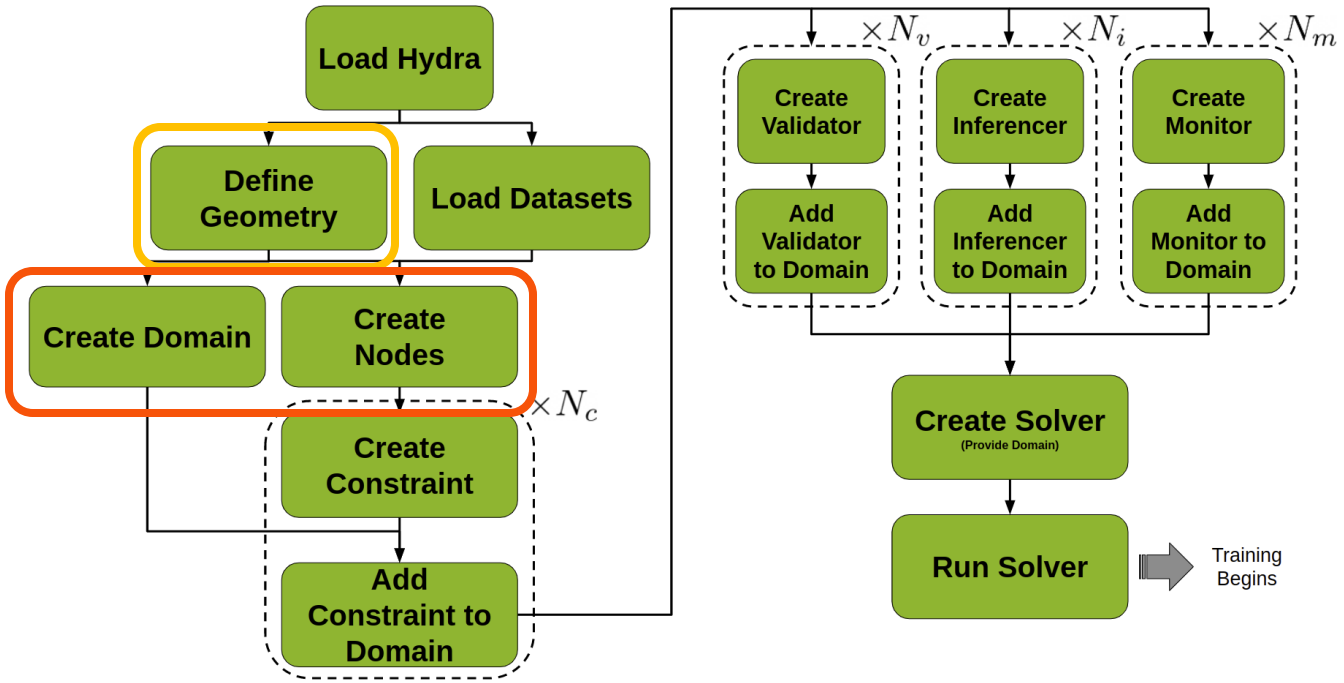
Load Hydra




```yaml
defaults :
  - PhysicsNeMo_default
  - scheduler: tf_exponential_lr
  - optimizer: adam
  - loss: sum
  - _self_

scheduler:
  decay_rate: 0.95
  decay_steps: 200

save_filetypes : "vtk,npz"

training:
  rec_results_freq : 1000
  rec_constraint_freq: 1000
  max_steps : 5000
```

PhysicsNeMo: Anatomy of a project

Create geometry, domain and nodes

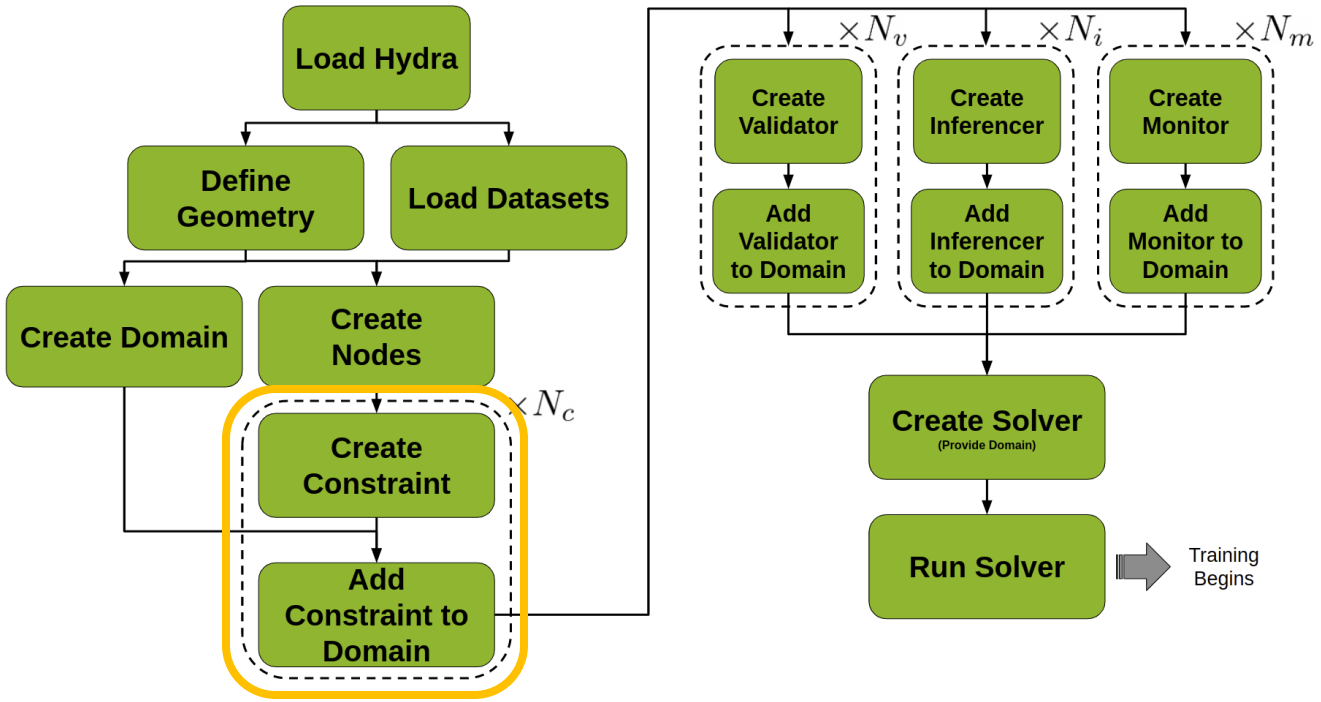



```python
@PhysicsNeMo.main(config_path="conf", config_name="config")
def run(cfg: PhysicsNeMoConfig) -> None:

    # make geometry
    x = Symbol("x")
    geo = Line1D(0, 1)
```

```python
    # make list of nodes to unroll graph on
    eq = CustomPDE(f=1.0)
    u_net = FullyConnectedArch(
        input_keys=[Key("x")],
        output_keys=[Key("u")],
        nr_layers=3,
        layer_size=32
    )

    nodes = eq.make_nodes() +
[u_net.make_node(name="u_network")]

    # make domain
    domain = Domain()
```

PhysicsNeMo: Anatomy of a project

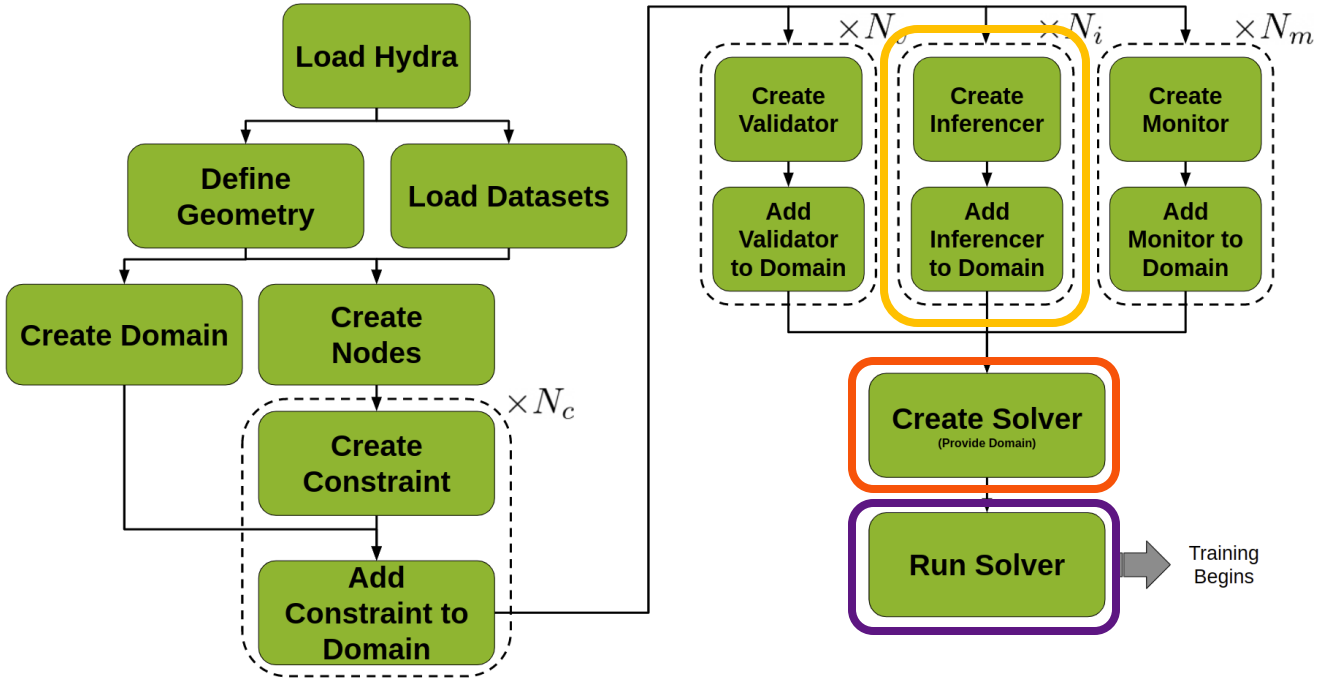
Add constraints




```python
# add constraints to solver

# bcs
bc = PointwiseBoundaryConstraint(
    nodes=nodes,
    geometry=geo,
    outvar={"u": 0},
    batch_size=2,
)
domain.add_constraint(bc, "bc")

# interior
interior = PointwiseInteriorConstraint(
    nodes=nodes,
    geometry=geo,
    outvar={"custom_pde": 0},
    batch_size=100,
    bounds={x: (0, 1)},
)
domain.add_constraint(interior, "interior")
```

PhysicsNeMo: Anatomy of a project

Create utils to visualize the results and run the solver




```
    # add inferencer
    inference = PointwiseInferencer(
        nodes=nodes,
        invar={"x": np.linspace(0, 1.0, 100).reshape(-1,1)},
        output_names=["u"],
    )
    domain.add_inferencer(inference, "inf_data")
```

```
    # make solver
    slv = Solver(cfg, domain)

    # start solver
    slv.solve()

if __name__ == "__main__":
    run()
```

```
python <script_name>.py
mpirun -np <#GPU> <script_name>.py
```

Solving Parameterized Problems

Problem definition

- Consider the parameterized version of the same problem as before. Suppose we want to determine how the solution changes as we move the position on the boundary condition $u(l) = 0$
- Parameterize the position by variable $l \in [1, 2]$ and the problem now becomes:

$$\mathbf{P}: \begin{cases} \dfrac{\delta^2 u}{\delta x^2}(x) = f(x) \\ u(0) = u(l) = 0 \end{cases}$$

- This time, we construct a neural network $u_{net}(x, l)$ which has x and l as input and single value output $u_{net}(x, l) \in \mathbb{R}$.
- The losses become

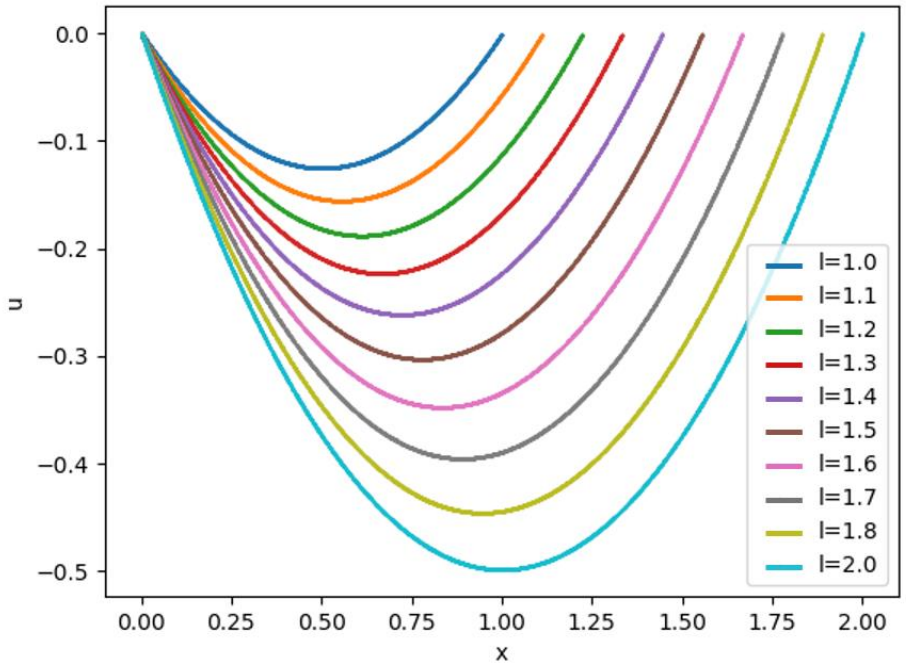
$$L_{Residual} = \int_1^2 \int_0^1 \left(\frac{\delta^2 u_{net}}{\delta x^2}(x) - f(x) \right)^2 dx dl \approx \left(\int_1^2 \int_0^1 dx\, dl \right) \frac{1}{N} \sum_{i=0}^{N} \left(\frac{\delta^2 u_{net}}{\delta x^2}(x_i, l_i) - f(x_i) \right)^2$$

$$L_{BC} = \int_1^2 \big(u_{net}(0, l)\big)^2 + \big(u_{net}(l, l)\big)^2 dl \approx \left(\int_1^2 dl \right) \frac{1}{N} \sum_{i=0}^{N} (u_{net}(0, l_i))^2 + (u_{net}(l_i, l_i))^2$$

Solving Parameterized Problems

Results

- For $f(x) = 1$, for different values of l we have different solutions



Solution to the parametric problem

Solving Inverse Problems

Problem definition

- For inverse problems, we start with a set of observations and then calculate the causal factors that produced them
- For example, suppose we are given the solution $u_{true}(x)$ at 100 random points between 0 and 1 and we want to determine the $f(x)$ that is causing it
- Train two networks $u_{net}(x)$ and $f_{net}(x)$ to approximate $u(x)$ and $f(x)$

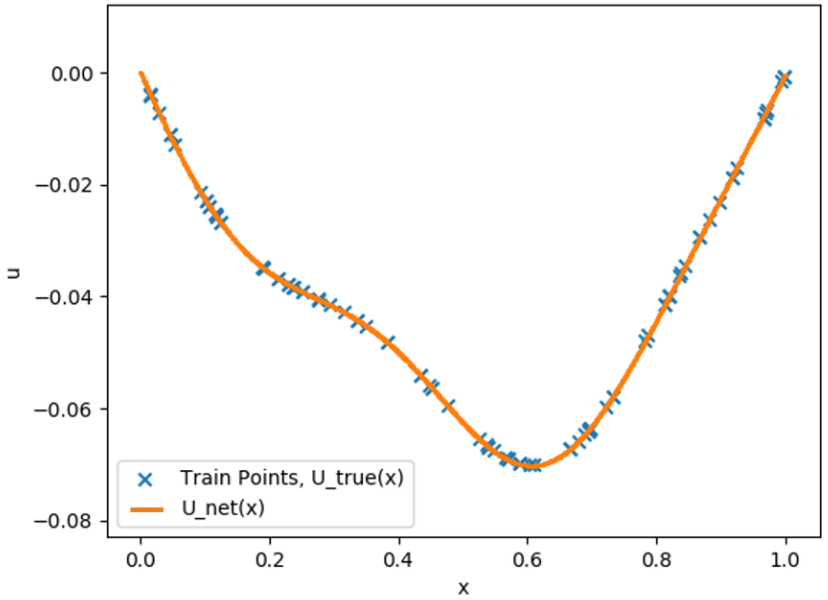
$$L_{Residual} \approx \left(\int_0^1 dx\right)\frac{1}{N}\sum_{i=0}^{N}\left(\frac{\delta^2 u_{net}}{\delta x^2}(x_i) - f(x_i)\right)^2$$

$$L_{Data} = \frac{1}{100}\sum_{i=0}^{100}\left(u_{net}(x_i) - u_{true}(x_i)\right)^2$$

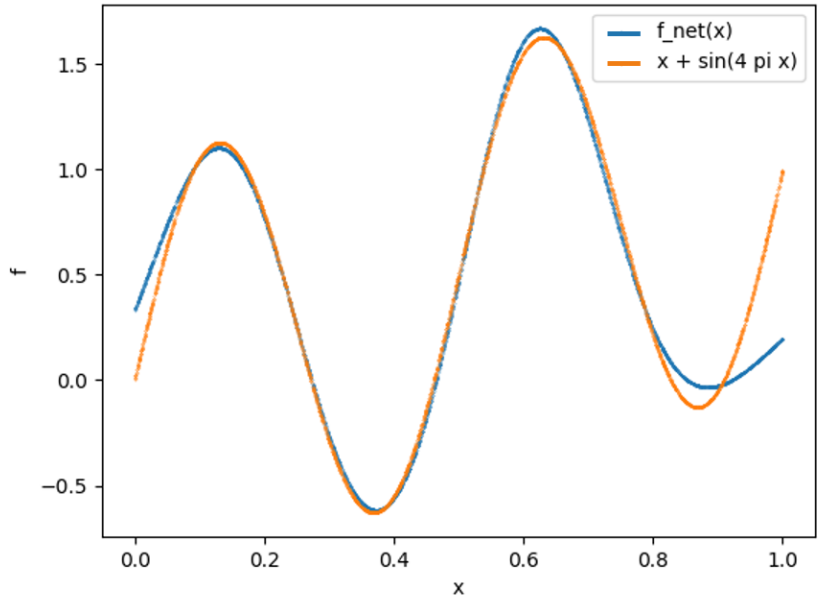
Solving Inverse Problems

Results

- For $u_{true}(x) = \frac{1}{48}\left(8x(-1+x^2) - \frac{3\sin(4\pi x)}{\pi^2}\right)$ the solution for $f(x)$ is $x + \sin(4\pi x)$



Comparison of $u\_net\ (x)$ and train points from u_true



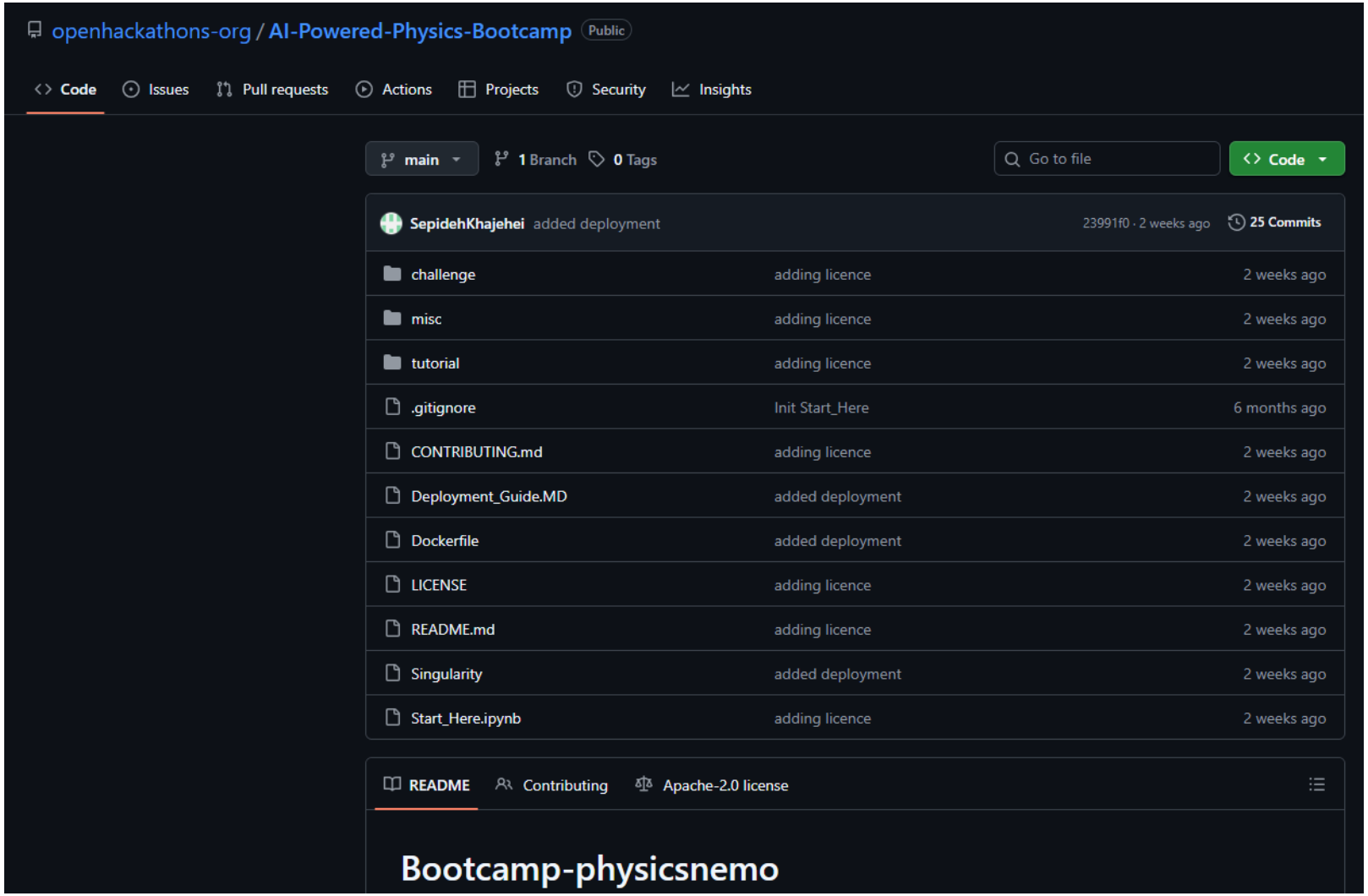
Comparison of the true solution for $f(x)$ and the $f_{net}(x)$ inverted out

Hands-on with PhysicsNeMo — tutorial

Bootcamp material, see short URL below

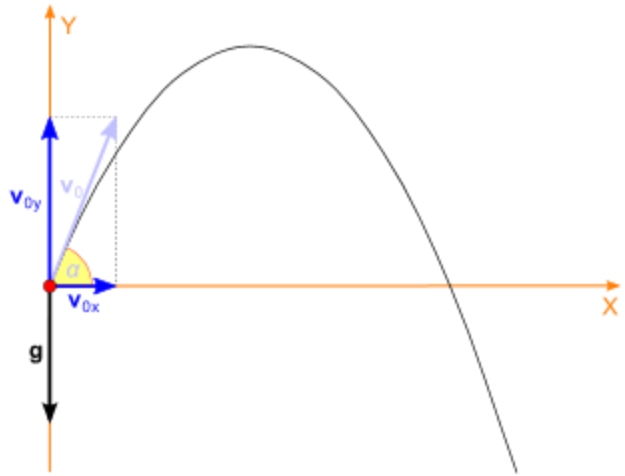
https://shorturl.at/Vf1Qn

(from long url: https://github.com/openhackathons-org/AI-Powered-Physics-Bootcamp)



Projectile motion

Problem description



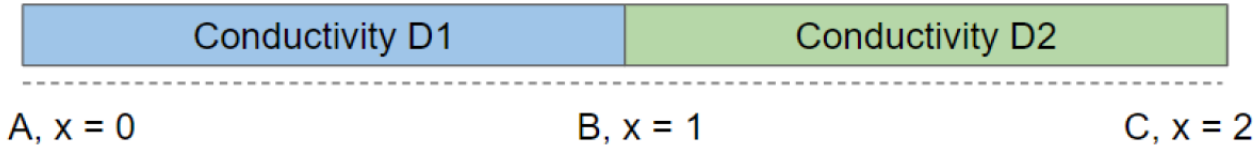
- Solving projectile motion in the case of absence of air-resistance on the surface on Earth.
- Equations: Equations of motions are as follows.

$$\frac{d^2}{dt^2} S_x = 0 \qquad \frac{d^2}{dt^2} S_y = -g$$

- Let us solve these two equations with the ball starting at origin (0,0)

1D diffusion

Problem description



- Composite bar with material of conductivity $D_1 = 10$ for $x \in (0,1)$ and $D_2 = 0.1$ for $x \in (1,2)$. Point A and C are maintained at temperatures of 0 and 100 respectively
- Equations: Diffusion equation in $1D$

$$\frac{d}{dx}\left(D_1 \frac{dU_1}{dx}\right) = 0 \qquad \text{When } 0 < x < 1$$

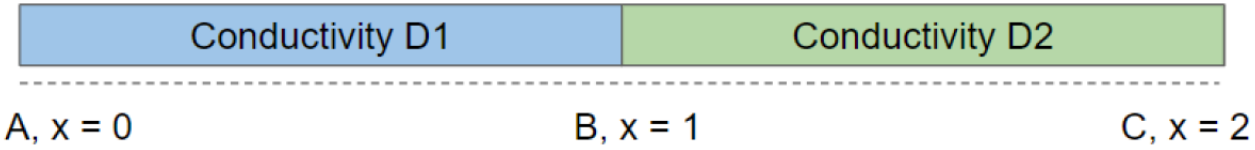
$$\frac{d}{dx}\left(D_2 \frac{dU_2}{dx}\right) = 0 \qquad \text{When } 1 < x < 2$$

- Flux and field continuity at interface (x=1)

$$\left(D_1 \frac{dU_1}{dx}\right) = \left(D_2 \frac{dU_2}{dx}\right)$$
$$U_1 = U_2$$

Parameterized 1D diffusion

Problem description



- Composite bar with material of conductivity D_1 for $x \in (0,1)$ and $D_2 = 0.1$ for $x \in (1,2)$.
- Solve the problem for multiple values of D_1 in the range $(5, 25)$ in a single training
- Same boundary and interface conditions as before
- Equations: Diffusion equation in $1D$

$$\frac{d}{dx}\left(D_1 \frac{dU_1}{dx}\right) = 0 \qquad \text{When } 0 < x < 1$$

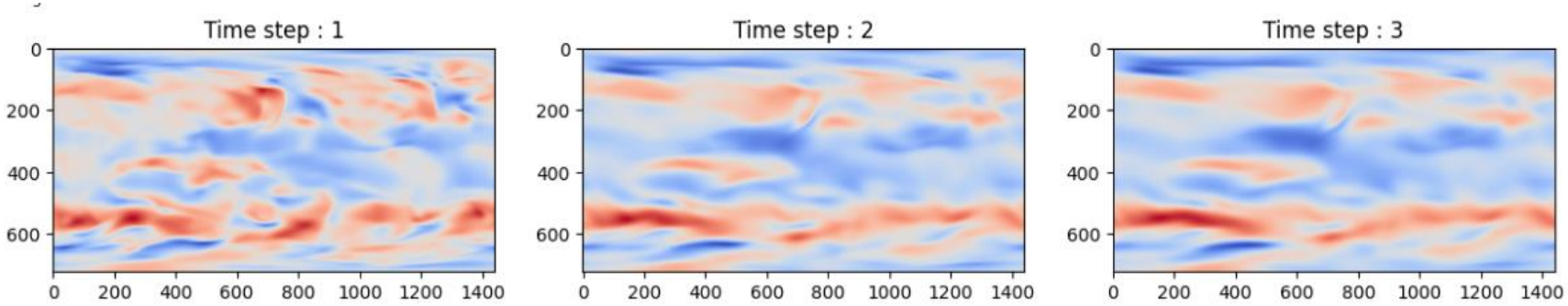
$$\frac{d}{dx}\left(D_2 \frac{dU_2}{dx}\right) = 0 \qquad \text{When } 1 < x < 2$$

- Flux and field continuity at interface (x=1)

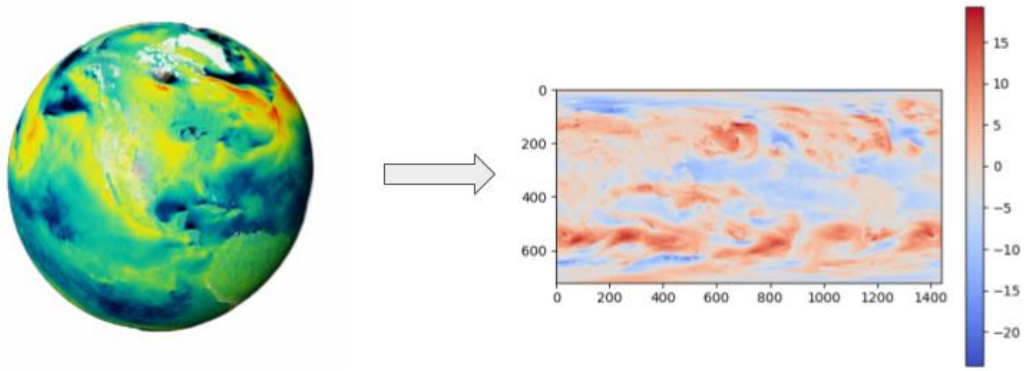
$$\left(D_1 \frac{dU_1}{dx}\right) = \left(D_2 \frac{dU_2}{dx}\right)$$
$$U_1 = U_2$$

Weather prediction using Navier-Stokes

Problem description



- We aim to predict the velocities for 6-hour timesteps using the Navier-Stokes equation.
- The Navier–Stokes equations mathematically express the conservation of momentum and conservation of mass for Newtonian fluids.
- We will take a 2d projected input from the ERA5 Reanalysis dataset to be used as initial conditions for our input. The process of taking the 3D sphere and projecting it onto a 2D mesh is shown in the diagram below, the 2D mesh is of the size (1440,720)



NVIDIA AI Technology Center (NVAITC)

NVIDIA AI TECHNOLOGY CENTER (NVAITC)

Outcome-Focused Collaborations for Research & Talent Development

NVAITC General Research Guidelines

Project Selection	• Jointly agreed by Host and NVAITC • NV criteria include technology stack, computing scale, novelty • an agreed-upon “statement of work”
PI Contribution	• Access to existing compute platform resources • Funding • Researchers
NV Contribution	• GPU/ML/DL technology / adoption enablement • Staff effort is bounded in both total time and duration • No joint IP • Beyond NVAITC, other NVResearch or teams may or may not collaborate as per any other academic collaboration
Expected Outcome	• scientific publication, with NVAITC acknowledgement or co-authorship • open-source code release

AI4Sience Bootcamp 活動交流時間登記表

- 感謝您參與本次活動！我們希望在活動期間 (**3/11 在 NCHC 13:30 ~ 15:30**) 安排短暫的一對一或小組交流，了解彼此，看未來是否有合作的可能。
- 由於可安排的交流時段有限，能實際安排到的組別名額不多。
- 若您在問題中提供**越完整、越具體的資訊**，我們就越容易判斷如何安排，增加與我們對談的機會。
- 送出表單後，我們會依時間與媒合情況，另行通知是否能安排交流時段。
- **表單填寫截止時間 : 3/10 下午 6 點**
- https://forms.gle/cTzbUTrpXuXEW5sq7

THANK YOU